Testing LLM-Serving Scheduler Ranking Portability with Trace-Driven Simulation

SOROUSH VAHIDI, New Jersey Institute of Technology, USA

Comparative conclusions from simulation studies can depend on the workload, operating condition, and metric used to produce them. We study this problem for large language model (LLM) serving schedulers through LSSP, a trace-driven deterministic discrete-event simulation study with a preregistered paired selection-and-comparison design. The evaluation uses three independently released workload sources, 120 workload windows, six calibrated operating regions, and a fixed scheduler panel, producing 9,360 unique policy–window–region configurations and 18,720 deterministic executions. Methodologically, LSSP treats ranking portability $R(W, M, L)$ as the simulation output of interest, combines practical-margin and confidence-interval criteria for uncertainty-aware reversal detection, and estimates the workload sample complexity required for ranking recovery. Cross-source rank agreement is strong between two sources (Kendall's τ_b up to 1.0) but lower when BurstGPT is involved (as low as 0.55); 3.6% of pairwise comparisons (36/990) show statistically supported reversals, all concentrated in one policy pair with BurstGPT on one side. Cross-metric portability is more limited: median rank correlation is $\tau_b = 0.419$ and top-policy agreement is 31.9%. A selected physical vLLM/GPU validation did not reproduce the simulator-predicted major reversal, while the stable-control ordering qualitatively persisted. Simulation-derived scheduler rankings should therefore be treated as conditional on workload, operating region, metric, and model fidelity.

CCS Concepts: • **Computing methodologies** → **Simulation evaluation**; • **Computer systems organization** → *Cloud computing*.

Additional Key Words and Phrases: LLM serving, scheduling, ranking portability, trace-driven simulation, performance evaluation

1 Introduction

Simulation studies are routinely used to compare scheduling policies when exhaustive real-system experimentation is too expensive, slow, or difficult to control. Such studies do not produce scheduler conclusions in the abstract: a comparative ordering is produced under a chosen workload, operating region, resource model, and evaluation metric. The same simulation campaign can therefore answer "which scheduler is better" only relative to the conditions under which that comparison was made. If the workload source, load region, metric, or simulator fidelity boundary changes, the comparative ranking may remain stable, degrade gradually, or reverse. This paper studies that portability problem as the primary output of a trace-driven simulation experiment, rather than treating it as a secondary sensitivity check after a single scheduler leaderboard has already been reported.

LLM serving is a useful and practically important setting for this question. Modern LLM serving targets high-performance, GPU-resident inference systems whose scheduling decisions govern how requests share costly accelerator, memory, and batching resources, and serving stacks have grown increasingly sophisticated in how they make those decisions. Beyond simple first-come-first-served execution, modern serving stacks employ continuous batching and paged key-value (KV) cache management [13], iteration-level scheduling [35], chunked prefill to balance prefill and decode work within a batch [2], disaggregated prefill/decode execution across dedicated hardware pools [19, 39], KV-cache-centric disaggregated architectures [20], dynamic instance-level rebalancing [24], explicit fairness objectives [22], output-length-aware admission and memory packing [12],

Author's Contact Information: Soroush Vahidi, sv96@njit.edu, New Jersey Institute of Technology, Newark, New Jersey, USA.

2026. ACM 2376-3647/2026/10-ART
https://doi.org/

and learning-to-rank decode-priority policies [9]. Each of these mechanisms was introduced, and evaluated, against a particular choice of simulation or experimental conditions: a specific trace, a specific arrival process, a specific mix of prompt and output lengths, a specific operating load, and a specific metric.

A separate line of work has documented that these workloads are themselves far from uniform. Public production traces from different services and time periods differ substantially in burstiness, request concurrency, prompt and output length distributions, and failure characteristics [19, 23, 31]; large-scale characterization studies of KV-cache reuse and of year-long production traffic report further axes of heterogeneity across clients, models, and time [17, 28]; and dedicated workload-generation frameworks exist precisely because no single fixed trace is thought to be representative of production traffic in general [33]. Individually, prior work has therefore already established two things: that LLM-serving mechanisms are numerous and mechanistically diverse, and that the workloads used to evaluate them are themselves heterogeneous and shift across source, time, and operating region.

What has not been directly asked for LLM-serving scheduler comparisons is the simulation-comparison question that follows from combining these two observations: whether comparative scheduler conclusions transfer across contexts. The typical evaluation question in this literature is: *how well does scheduler A perform, relative to scheduler B, on the workload(s) chosen for this paper's experiments?* Because a simulation study necessarily uses only a finite, chosen set of heterogeneous workloads, a comparative claim of this form is only as useful as its *portability*: does the same comparative ordering of schedulers hold if the workload distribution, the operating load, or the evaluation metric is changed to another one a practitioner might reasonably care about? This question differs both from workload heterogeneity itself and from any single scheduler's sensitivity to load or service-level objectives (SLOs): it concerns the reliability of the *comparison*, not of any individual system. LSSP is designed to measure this transferability directly under a fixed simulation protocol.

We formalize this as the portability of a comparative scheduler ranking $R(W, M, L)$, where W denotes a workload distribution, window, or source; M denotes an evaluation metric; and L denotes an operating or load region. $R(W, M, L)$ is the ordering (full or partial) that a fixed panel of scheduling policies induces as the comparative output of a simulation experiment under a fixed protocol, holding W, M, and L at specified values. Ranking portability asks how R behaves as W, M, or L is varied: whether the ordering of policies is preserved, where and how it reverses, and how much evidence (how many independent workload windows, drawn from how many sources) is required before a reported ranking can be trusted to generalize beyond the exact windows used to produce it.

This question matters because a scheduler comparison is scientifically and practically useful only insofar as its comparative conclusion transfers to the workload distributions a reader actually cares about. A benchmark result of the form "scheduler A outperforms scheduler B" is not a claim about A and B in the abstract; it is implicitly a claim about A and B *under the workload, metric, and load conditions the benchmark used.* If that comparative ordering is fragile under source, metric, or load-region substitution, then benchmark results risk being read as more general than they are, and evaluation effort risks being spent on a single workload family when the resulting conclusion may not transfer. Conversely, if comparative orderings are portable across independent sources, metrics, and load regions, that is itself a useful, non-obvious finding: it would mean a leaner benchmark protocol is sufficient for future scheduler comparisons, and it would give scheduler designers a much stronger warrant for claims of general improvement.

We refer to the benchmark built around this question as the LLM-Serving Scheduler Portability Benchmark (LSSP). LSSP asks the following research questions, fixed under a preregistered protocol before any comparative outcome was generated (§3):

RQ1. How stable are scheduler rankings across independent workload sources?
RQ2. How stable are scheduler rankings under temporal, provider, and domain shifts?
RQ3. To what extent do rankings obtained on synthetic stress workloads transfer to rankings on independent real-trace-derived workloads?
RQ4. Which source-native observable workload characteristics are associated with cross-distribution scheduler rank reversals?
RQ5. How many independent workload windows are required before a comparative scheduler ranking becomes statistically stable?
RQ6. Do representative simulated scheduler rankings and cross-workload rank reversals reproduce on a real serving engine?

Load-level dependence is deliberately treated as a secondary robustness/sensitivity axis applied to RQ1–RQ4, rather than as a headline research question in its own right, to avoid duplicating a closely related prior load-scaling study on an overlapping window set (§3).

Contributions. This study executes that preregistered protocol and reports its structurally validated results. This work contributes:

- a formulation of comparative scheduler-ranking portability, $R(W, M, L)$, as an object of simulation-based selection and comparison studies, distinct from both workload characterization and single-scheduler evaluation;
- a preregistered paired experimental design for comparing a fixed panel of scheduling alternatives across independently sourced workload windows, calibrated operating regions, and metrics, so that ranking-portability statistics are computed on matched observations rather than separately tuned per-source experiments;
- a deterministic, trace-driven discrete-event simulation execution model for LLM-serving scheduler comparisons, with repeated executions used as verification of bit-identical outcomes rather than as stochastic replicates (§5.3);
- an uncertainty-aware reversal-detection methodology that distinguishes practically meaningful rank reversals from microscopic sign changes, using preregistered practical-margin and confidence-interval criteria, and that treats every completion-conditioned metric's undefined cases explicitly rather than by imputation (§5);
- a benchmark sample-complexity analysis quantifying how many independent workload windows are required before a comparative ranking reproduces, applied to this study's own frozen 120-window corpus (§7);
- a characterization of the workload corpus underlying the simulation study, establishing that the three sources are measurably different in observed descriptor distributions and strongly separable under a leakage-resistant, chronologically blocked evaluation (§4);
- a selected-case physical validation on real vLLM/GPU hardware (RQ6, §8) used explicitly to characterize simulator fidelity and domain of validity for the benchmark's largest identified simulated reversal and a paired stable-control comparison; and
- a public artifact release of the benchmark's frozen workload identities, policy panel, protocol, simulation outputs, and analysis artifacts, to support independent reproduction (§11).

Comparative rankings prove neither uniformly stable nor uniformly unstable, but source-pair- and load-region-dependent (Sections 6–8); we do not claim that any scheduler in the panel wins in general (§9).

2 Related Work

2.1 Serving Architectures and Scheduling Mechanisms

A substantial body of systems work has improved LLM inference throughput and latency through scheduling and memory-management mechanisms. Orca introduced iteration-level scheduling at the granularity of individual decoding steps [35]. vLLM introduced PagedAttention, a paged KV-cache allocator that raised achievable batch sizes and became a widely adopted serving-system reference point [13]. Sarathi-Serve interleaves chunked-prefill computation with ongoing decode steps to reduce head-of-line blocking under continuous batching [2]. Splitwise and DistServe disaggregate the compute-bound prefill phase and the memory-bound decode phase onto separate hardware pools [19, 39]; Mooncake extends this direction with a KV-cache-centric disaggregated architecture that schedules around a shared, elastic KV cache [20]. Llumnix adds dynamic cross-instance request migration and rebalancing [24], and DynamoLLM reconfigures an inference cluster's resources for energy and cost efficiency under latency SLOs [23]. For geographically distributed LLM inference, Sun et al. [25] study block placement and request routing over heterogeneous GPU servers, combining experimentally validated performance models with optimization and simulation. That work is directly adjacent to LSSP in treating LLM inference as a resource-allocation performance problem, but its object is placement and routing optimization for one distributed serving architecture rather than the portability of scheduler rankings across independent workloads and metrics.

A second cluster of work targets scheduling *policy* design more specifically. VTC (Virtual Token Counter) defines an explicit token-level fairness objective for continuous-batching schedulers with a provable service-difference bound [22]. S^3 predicts each request's output length to pack KV-cache memory more tightly at admission time [12], whereas Fu et al. [9] learns a pairwise ranking model over waiting requests to prioritize decode order. Recent systems diversify the mechanism space further along four lines: preemptive, admission-control, and SLO-aware scheduling under imprecise length information or heterogeneous per-request SLOs, as in FastServe, JITServe, and SCORPIO [26, 32, 38]; learned-ranking, multi-objective, and single-score policies, as in PARS, the SLAI/RAD family, and LMetric [4, 27, 37]; request partitioning, hybrid KV-cache/hidden-state representations, and multi-priority engine/service-layer scheduling, as in Libra, Apt-Serve, and ProServe [10, 11, 21]; and tail-latency, multi-tenant, and agentic scheduling, including prediction-free interpolation between first-come-first-served and shortest-job-first with cache-aware preemption [14], hierarchical multi-agent scheduling under non-stationary multi-tenant load [15], and session-centric routing that trades load balance against cache locality [29]. Each of these papers reports an improvement over one or more baselines on the workload(s) chosen for its own experiments; none asks whether the *relative ordering* it reports would be preserved under an independently sourced workload, a different metric, or a different operating load – the question this benchmark is built around (§2.6).

Classical and data-center scheduling work offers useful context for the distinction between designing a scheduling policy and evaluating the robustness of scheduler conclusions. For example, Chen et al. [5] design and analyze non-preemptive policies for multiserver jobs and evaluate them on data-center traces, addressing the mean-response-time consequences of high-utilization scheduling decisions. LSSP does not propose a new scheduler in that line; it asks whether comparative scheduler-performance conclusions remain stable when the workload source, operating region, or metric changes.

2.2 Production Workloads and Characterization

A parallel literature documents that LLM-serving workloads observed in production are themselves heterogeneous. BurstGPT releases 10.31 million traces collected from a regional deployment of

Microsoft's Azure OpenAI GPT service over 213 days, characterizing request burstiness, conversation structure, and response-length behavior, and is now published as a full KDD 2025 paper rather than only as a preprint [31]. Two official Microsoft releases document Azure's own internal LLM-inference traffic: Splitwise analyzes production conversation and code-completion traces collected on Azure in November 2023 [19], and DynamoLLM analyzes a longer, one-week Azure trace collected in May 2024 [23]. Wang et al. [28] characterizes KV-cache reuse patterns at a large cloud provider, finding that reuse probability and lifespan are workload-category-dependent in ways that synthetic workload studies had not captured. Nixon et al. [17] reports a one-year longitudinal characterization of production traffic spanning workload evolution, caching behavior, and load balancing (a recent preprint at the time of writing; §2.6). ServeGen instead characterizes a worldwide production inference service in order to build a principled per-client workload *generator*, motivated explicitly by the position that no single fixed trace is representative enough on its own [33]; because it characterizes the same underlying Alibaba Cloud platform as this project's Bailian/Qwen source, we do not count it as an independent workload provider (§4). TraceLab characterizes a distinct workload family entirely – coding-agent sessions rather than single-turn or short-multi-turn chat requests – and is used here only as out-of-distribution context, not as one of this project's three primary sources.

2.3 Simulation and Benchmark Frameworks

Vidur is the closest existing infrastructure precedent to LSSP: a large-scale simulation framework purpose-built for exploring LLM-inference system configurations at a fidelity and speed unavailable from running full experiments on real hardware [1]. LLMServingSim and its successor LLMServingSim 2.0 provide a related but architecturally different hardware/software co-simulation infrastructure, adding iteration-granularity simulation with algorithmic-redundancy reuse and, in the 2.0 revision, explicit support for heterogeneous accelerators and disaggregated serving techniques [6, 7]. Two recent 2026 benchmarking frameworks expand this domain: CCL-Bench 1.0 [8] introduces a diagnostic, trace-based evaluation of LLM training and serving infrastructure to record fine-grained execution evidence of kernels and communication events, while XPerf [30] benchmarks serving systems under agentic workloads by employing fine-grained trace replay and full-stack profiling to handle agentic nondeterminism. These simulators and benchmarking frameworks are built to let researchers explore serving-system design points cheaply or load-test systems under agentic dependencies. LSSP uses the same general logic that makes simulation valuable – controlled comparison under repeated, matched conditions – but asks a different methodological question: whether the *ranking* induced by a fixed simulation protocol is portable across independently sourced workloads, operating regions, and metrics. LSSP's own simulator (§5) is project-local infrastructure, not a contribution positioned against this literature; we make no claim that its output is equivalent to production hardware latency or throughput, only that it supports a fixed, controlled benchmark protocol for relative comparisons and selected simulator-fidelity testing (§8).

2.4 Workload Dependence and Benchmark Sensitivity

Prior work has already established, individually, that LLM-serving workloads are heterogeneous (§2.2) and that individual serving mechanisms and schedulers are sensitive to batching structure, prefill/decode balance, load level, and service-level objectives (§2.1). We treat both of these as accepted background, not as a novel contribution of this work. Closest to this paper's motivation, Papaioannou and Doudali [18] show that a serving system's measured performance and the relative standing of design choices can themselves shift depending on which workload is used to evaluate them, arguing for workload-choice awareness as a first-class methodological concern;

that result is stated for single-system evaluation. In classical data-center dispatching, Yildiz et al. [34] compare Round Robin, Join-Idle-Queue, and Least-Work-Left on Google ClusterData v3 and Alibaba Cluster Trace v2018 using analytical models and trace-driven simulation. Their results show that production workload structure and the job-level versus task-level trace representation can change dispatching-policy behavior and, in some production-trace cases, the policy ordering. This is the closest non-LLM precedent for the evaluation concern LSSP studies. The distinction is that Yildiz et al. diagnose model–trace discrepancies and dispatcher behavior for data-center workloads, whereas LSSP makes the portability of a *comparative ordering* among a fixed panel of schedulers the primary benchmark object, asking whether that ordering is preserved across independently sourced workloads, operating regions, and metrics in LLM serving – the distinction this paper's Introduction develops between single-system sensitivity and comparative-ranking portability.

2.5 Ranking Portability, Comparison, and Benchmark Methodology

The statistical methodology this benchmark uses – Kendall's tau-b and Spearman's rho for pairwise ranking agreement, paired comparison and block-bootstrap confidence intervals for ranking-uncertainty quantification, and a probability-of-correct-selection framing for benchmark sample complexity – draws on standard nonparametric statistics and comparative-algorithm-ranking methodology (§5). In this paper those tools are used as a simulation-based selection-and-comparison procedure: the object being compared is not a new scheduler, but the stability of the comparative ordering produced by a fixed simulation design. The closest methodological precedent we identified is from the classical parallel-job- scheduling literature: Zakay and Feitelson [36] partition a real scheduler workload trace into independent per-user subtraces and recombine them to resample varied-but-realistic workloads, evaluating how a *single* parallel job scheduler's measured performance responds to workload resampling. LSSP's sample-complexity question (§7) is the same evaluation instinct – using resampling to ask how much workload evidence a scheduler-performance conclusion actually rests on – applied to a different object: the portability of a *comparative* ranking among multiple schedulers across independently sourced workloads, rather than one scheduler's sensitivity to resampled variants of a single trace.

2.6 Position of LSSP

Prior work generally studies a scheduler, a serving system, a simulation environment, or a workload generator/characterization in its own right. LSSP instead treats the portability of *comparative* scheduler rankings as the primary object of study across independently sourced LLM-serving workload sources, operating regimes, and metrics, using a fixed, mechanism-diverse scheduler panel and a preregistered protocol, while explicitly quantifying ranking uncertainty and benchmark sample complexity. A primary-source literature search targeting exactly this question (scheduler-ranking stability/portability, cross-workload scheduler evaluation, benchmark sample complexity and representativeness for LLM-serving systems, current through September 2026) did not surface prior work with this primary object. The closest precedents differ in object: Yildiz et al. [34] show on Google and Alibaba data-center traces that workload structure and trace granularity can affect dispatcher behavior and policy ordering, but do not study LLM-serving schedulers, cross-metric ranking portability, supported reversal analysis, or benchmark sample complexity; Zakay and Feitelson [36] (§2.5) evaluate one scheduler's sensitivity to workload resampling, not comparative ranking across schedulers; Vidur and LLMServingSim are simulation infrastructure (§2.3); and the production-characterization literature (§2.2) establishes the workload heterogeneity this design responds to without studying cross-source ranking portability. This wording is intentionally narrower than an unconditional claim of priority: we do not claim to be the first to study LLM-serving

workload dependence or scheduler sensitivity, only that we did not identify a prior benchmark treating cross-source, cross-metric, cross-load-region scheduler-ranking portability as its primary object of study.

3 Study Design

3.1 Research Questions

The six research questions defined in §1 were frozen before outcome generation. The primary confirmatory evidence is RQ1/RQ2 cross-source and temporal ranking portability on the frozen real-trace corpus. Provider and domain variation are not clean causal isolations: BurstGPT and Azure ultimately share Microsoft/Azure infrastructure (§4.1), and domain shift is addressed only through the cross-source contrasts reported in §6. RQ3 is secondary and represented here by an engineering-validation pilot of the synthetic-to-real transfer pipeline, not by a full confirmatory transfer result on the same footing as RQ1/RQ2. Load-level dependence is likewise a secondary sensitivity analysis applied to RQ1–RQ4 (§3.3).

3.2 Study Overview

LSSP evaluates a fixed panel of 13 scheduling policies (§5.1) against a frozen corpus of 120 workload windows drawn from three independently sourced traces (§4.1), across a calibrated six-region grid of operating regions (§5.6) and a fixed set of primary and completion-conditioned metrics (§5.7). Every policy is run against every window under every operating region, with a deterministic rep0/rep1 verification pass (§5.8), producing 9,360 unique policy–window–region configurations, each executed in two deterministic verification passes (18,720 total executions), from which per-source, per-region, and per-metric scheduler rankings are derived. Ranking-portability statistics (Kendall's tau-b, Spearman's rho, top-k overlap, and pairwise reversal detection) are then computed across pairs of sources, regions, and metrics, using workload window as the statistical unit (§3.4). This design was frozen (§3.5) before any comparative scheduler outcome existed.

3.3 Evidence Independence / Publication Boundaries

This project shares a lineage and a simulator codebase with a related manuscript that could otherwise appear to overlap in evidence, and its design was explicitly checked against it for overlap before the protocol was frozen:

- **"The Exploitability Gap in LLM-Serving Scheduler Portfolios" (LLM 2026).** That manuscript's own experiment used a fixed 60-window subset of public BurstGPT and Azure-2023 traces, a different load-factor grid, and an 8-policy portfolio; it is cited here strictly as prior motivation and never restated as this project's own evidence. This project's window sampling (§4.3) was independently constructed and verified non-identical to that 60-window set, and its load-level dependence analysis (§3.1) uses this project's own calibration protocol (§5.6), never that load grid.
- **A separate module-intervention manuscript ("Module Counterfactuals for Attributing LLM Serving Scheduler Effects").** This study makes no module-counterfactual or module-attribution claim; RQ4's workload-descriptor analysis (§5) is an offline, descriptive association analysis, not a deployed selector or an intervention-based attribution method.

3.4 Experimental Unit and Paired Design

The statistical unit of analysis is the workload window: a fixed-size (200-request) contiguous slice of a source trace, uniquely identified and frozen before any scheduler outcome existed (§4.3). Every policy in the panel (§5.1) is run against every window under every calibrated operating region

(§5.6), so that ranking-portability statistics are computed on genuinely paired observations – the same window, region, and metric definition, varying only the policy – rather than on separately tuned per-source experiments. Source, operating region, and metric are treated as factors of interest; policy identity is the object whose relative ordering is measured.

3.5 Preregistration

The workload window set (120 windows: 40 per source, 10 reused from the earlier discriminability pilot plus 30 newly sampled per source; §4.3), the scheduler panel and its selection rule (§5.1), the metric contract (§5.7), the six-region operating-load calibration (§5.6), and the ranking-portability analysis plan (reversal-detection thresholds, bootstrap procedure, sample-complexity design) were each frozen, and structurally validated, before any comparative scheduler outcome was generated. The exact frozen content is recorded in the public artifact's protocol documentation (§11).

4 Workload Corpus and Characterization

4.1 Sources

LSSP's primary benchmark corpus draws from three independently released workload sources: BurstGPT [31], Microsoft's official Azure 2024 LLM-inference trace [16, 23], and Alibaba Cloud's Bailian/Qwen anonymized usage-trace release [3]. Azure's 2023 conversation/code traces [19] and windows from an earlier discriminability pilot (§5.5) contribute additional context but are not part of the primary 120-window matrix beyond the 10-per-source pilot windows reused as described below. TraceLab [40] is retained as an out-of-distribution, coding-agent workload family for future expansion, not as one of the three primary sources, because its independence from a prior 512-window public sweep cannot be fully verified from public artifacts alone.

BurstGPT's own reported collection methodology logs traffic from a regional customer of Microsoft's Azure OpenAI GPT service, and this project's Azure trace is Microsoft's own official release of internal Azure LLM-inference-service traffic. Both therefore sit on the same underlying cloud provider's infrastructure. We describe BurstGPT and Azure as distinct, independently released workload sources – different collection methodology, different customer population, different release provenance and version history – and we do not describe them as independent providers. Bailian/Qwen (Alibaba Cloud) is a genuinely different underlying platform from both.

4.2 Representation and Provenance

Each source is ingested through a dedicated, field-level-audited adapter that maps the source's native schema onto a common internal request representation (arrival time, input tokens, output tokens, total tokens, session identifier where available, model class). No field is fabricated where a source's native schema lacks it (for example, BurstGPT's documented schema has no tenant, KV-reuse, SLO, or failure-code field, and none is synthesized). License status for each source was verified before any confirmatory use of a real, full-scale sample (for example, BurstGPT is under CC-BY-4.0).

4.3 Window Construction

The benchmark corpus freezes exactly 120 windows of 200 requests each: 40 per source (`azure_llm_2024`, `bailian_qwen`, `burstgpt`), each composed of 10 windows reused verbatim from the discriminability pilot (§5.5) plus 30 new windows drawn by a deterministic, outcome-blind, source-symmetric pinned-extension sampler that uses only valid-row index ranges and fixed seeds – never scheduler metrics, policy identities, or the pilot's tie/non-tie outcomes (verified by structural audit to contain no BurstGPT-outcome-targeting signal). Windows are assigned to EARLY/MIDDLE/LATE

chronology strata by deterministic relative chronology (60/30/30 windows respectively); the reused pilot windows are all EARLY. The freeze was independently validated – unique window identities, exact per-source and reused/new counts, fixed `request_count = 200`, no prohibited overlap, valid chronology, and absence of any scheduler/policy/load-calibration outcome field in the selection rule – and recorded with a canonical content hash (excluding generation timestamp; full values and verification records are in the public artifact hash ledger) before any load calibration or scheduler outcome was generated.

4.4 Descriptors

Each window is described by a 28-feature descriptor vector spanning prompt- and output-token-length statistics (mean, p90, long-prompt fraction at the 512/2048/8192-token thresholds), arrival-timing statistics (mean arrival rate, inter-arrival percentiles), and derived load-pressure proxies. Three deterministically redundant pressure-proxy features ($R^2 = 1.0$ against other included features) are retained in the underlying dataset but excluded from the classifier input used in §4.6. Prompt-token figures compose differently across sources for architectural reasons: TraceLab's and Azure's prompt-token fields are potentially multi-turn-cumulative context, while BurstGPT's is a single per-request figure. This is a property of how each source's underlying workload is architected (a growing agent/conversation session versus a single stateless API call), not an adapter artifact, and it is preserved as an explicit caveat on every prompt-length-based claim below.

4.5 Cross-Source Differences

This characterization analysis uses a separate, larger population than the primary 120-window ranking corpus: 100 windows (window size 200) drawn from each of *four* source classes – `azure_llm_2024`, `bailian_qwen`, `burstgpt`, and `tracelab` (§4.1) – for $n = 400$ windows total. TraceLab is included here only as a fourth descriptor class to stress-test the classifier against a maximally distinct, out-of-distribution workload family; it is not one of the three primary ranking-portability sources and contributes no windows to the 120-window corpus used in §6–§6.7. Under a leave-one-time-bucket-out (EARLY/MIDDLE/LATE), grouped, chronologically blocked cross-validation over these $n = 400$ windows, a random forest classifier distinguishes the four descriptor-labeled source classes with pooled balanced accuracy 1.000 (mean ± std across three folds: 1.000 ± 0.000); a multinomial logistic regression reaches 0.988 balanced accuracy (0.987 ± 0.007 across folds); and a depth-limited (depth $\leq$ 3) decision tree reaches 0.980 balanced accuracy (0.980 ± 0.018 across folds, worst single fold 0.955 with LATE held out). This grouped, chronologically blocked evaluation supersedes an earlier random-split result (which agreed at balanced accuracy 1.0) for any manuscript claim: random splitting is vulnerable to within-window-adjacent-time leakage, which the grouped evaluation is structurally immune to and was verified to prevent across the separability pipeline.

4.6 Source Separability

The workload sources are confirmed measurably separable, with the caveats below. The safe claim supported by this evidence is that the workload sources' observed descriptor distributions are measurably different, and that source identity is strongly separable under a leakage-resistant, chronologically blocked evaluation. This is not evidence that the underlying providers' conversations or tasks intrinsically differ in length or structure, only that their *observed, request-level* descriptor figures do, for a mixture of architectural and possibly-genuine-content reasons that this classifier-based analysis cannot separate (§4.4).

4.7 Drivers of Differences

An attribution analysis triangulating held-out permutation importance, single-feature balanced accuracy, leave-one-feature-out deltas, and feature-group-ablation deltas (all under the same grouped chronological cross-validation) identifies two tiers of driving features. Tier 1 – incrementally load-bearing, with nonzero permutation importance and a measurable accuracy cost when the `prompt_length` feature group is removed (−0.0051) – consists of `long_prompt_fraction_512`, `long_prompt_fraction_2048`, and `prompt_tokens_mean`. Tier 2 – individually strong single-feature classifiers (balanced accuracy up to 0.980) whose removal, individually or as a whole feature group, costs 0.000 accuracy because their signal is redundant with Tier 1 – includes total/prompt-token percentile features, inter-arrival percentiles, and mean arrival rate. Prompt-length structure, specifically the fraction of long prompts at the 512- and 2048-token thresholds and mean prompt length, is therefore the classifier's primary, non-redundant predictive feature for source separability, in the ablation sense defined above (largest classifier accuracy cost when removed) – not a claim that prompt length is the true underlying cause of the sources' differences, which may also reflect the multi-turn-cumulative-vs-single-request token-accounting confound already noted in §4.4; arrival-timing structure and output length are individually predictive but carry no additional signal once prompt-length features are present. A critical-ablation check confirms this separability is broad rather than dependent on any single feature: removing the two strongest Tier-1 features together changes random-forest balanced accuracy by at most −0.0025.

4.8 Window-Size Sensitivity

The benchmark corpus uses a fixed window size of 200 requests throughout, matching the discriminability pilot's and the LLM 2026 comparison's window size for methodological consistency. A dedicated window-size sensitivity sweep – varying the number of requests per window and re-checking separability/discriminability, distinct from the frozen `WINDOW_SIZE_SENSITIVITY` robustness-contract item (§6.5), which despite the shared name concerns how many *windows* are subsampled, not requests per window – has not yet been run; this is noted as an open item in §10 rather than a result reported here.

5 Simulation and Scheduler-Comparison Methodology

5.1 Scheduler Panel

The PRIMARY panel comprises 11 policies, executed alongside 2 additional `STYLE_APPROXIMATION` policies used only for a mandatory robustness check (13 policies executed in total): a classical arrival-order baseline (`fifo`, also the load-calibration reference policy), a deadline-aware policy (`edf`), a slack/urgency-aware policy (`least_laxity_first`), a service-length-aware policy (`estimated_service_time_first`), a fairness-aware policy (`weighted_fair_share`), a KV/memory-pressure-aware policy (`kv_constrained_online`), an admission-control/SLO-guard policy (`admission_control`), and faithful reimplementations of four external mechanisms: vLLM-style token-budget continuous batching in both its pre-chunked-prefill and chunked-prefill forms (`vllm_faithful`, `vllm_chunked_prefill_faithful`), Sarathi-style chunked prefill (`sarathi_faithful`), and an SLO-aware scheduling policy in the SLAI/RAD family (`slai_faithful`). The two `STYLE_APPROXIMATION` robustness policies are a token-budget-proxy style approximation (`vllm_style_token_budget`) and an SLO-guard-proxy style approximation (`scorpio_style_slo_guard`); two further faithful reimplementations requiring disaggregated/cross-instance simulator semantics (`distserve_faithful`, `llumnix_faithful`) are executed only in a secondary stratum, never

pooled with the single-GPU colocated PRIMARY leaderboard, and one candidate (`apt_serve_faithful`) is excluded entirely as `scaffolding_only` – not a validated reimplementation at the time of this freeze.

A policy enters the PRIMARY panel only if, at the time it was added to the comparability audit – frozen a full day before the discriminability pilot (§5.5) executed – it satisfies an outcome-independent selection rule: its mechanism predates this protocol's freeze, it occupies a mechanism family not already saturated by an existing panel member, it has adequate implementation fidelity and passes its own unit tests, it is simulator-semantically compatible with the single-GPU colocated execution model, and it is independently reproducible from a cited reference. No policy was added to or removed from the panel in response to the pilot's outcome, and mechanism families it identified as under-implemented (preemptive/multilevel scheduling in the FastServe sense, JITServe, LMetric) were deliberately left as documented future-work candidates rather than fast-tracked into the panel after seeing the pilot's BurstGPT collapse.

5.2 Fidelity Taxonomy

Every executed policy is labeled with one of five fidelity classes: `FAITHFUL_EXTERNAL` (a validated reimplementation of a specific published mechanism), `REPOSITORY_NATIVE_CLASSICAL` (a standard scheduling discipline implemented natively in this simulator, not claimed as a reproduction of any specific paper), `SIMULATOR_PROXY` (a mechanism-motivated policy without a single canonical external paper), `STYLE_APPROXIMATION` (an approximate, style-level proxy for a published mechanism, explicitly excluded from the PRIMARY headline panel and used only for robustness checks), and `scaffolding_only` (an unaudited, not-yet-validated implementation, excluded entirely). We do not claim that any `STYLE_APPROXIMATION` or `SIMULATOR_PROXY` policy is an official or validated reimplementation of its namesake published system.

5.3 Execution Model

Each (window, policy, operating-region) cell is executed as a deterministic discrete-event simulation. The entities are requests, one or more simulated GPU resources, a waiting queue, and policy-controlled active batches. Each request carries an arrival time, prompt-token count, target output-token count, priority weight, and, where required by a policy or metric, an SLO/deadline value. Simulator state includes the pending-arrival list, waiting queue, active requests per GPU, request phase and progress (prefill remaining, decoded tokens, first-token time), current KV-token footprint, per-step active-batch and utilization histories, completed and dropped requests, and observational mechanism telemetry (§5.9).

Time advances in fixed serving steps. At each step, all requests with arrival time less than or equal to the current simulation time are enqueued; the simulator exposes an observable state to the scheduling policy; the policy returns an action specifying admissions and, for policies that support them, preemption, swapping, migration, held decodes, or prefill-control decisions; the simulator validates the action against the resource model and applies it; active requests then advance by one service step; completed requests are removed and recorded; and per-step utilization, batch, KV, and contention histories are updated. The primary benchmark uses the colocated single-GPU-compatible stratum of this model; disaggregated or cross-instance semantics exist for secondary policies but are not pooled with the PRIMARY leaderboard (§5.1).

The resource model includes active-sequence capacity, batch/token-budget constraints where a faithful policy uses them, and KV-memory accounting. KV occupancy is tracked as demand relative to the configured nominal `max_kv_tokens`; only KV-aware policies enforce this limit at admission, so occupancy above 1.0 is interpreted as demand exceeding nominal capacity rather than as an instrumentation failure. In the default backward-compatible mode, prefill is instantaneous and each

Table 1. Main simulation-model scope and assumptions used in the PRIMARY ranking-portability study.

Model component	Treatment in this study
Arrivals	Trace-driven request arrivals from frozen 200-request windows.
Service progression	Fixed-step serving model; decode emits one token per active decoding request per step.
Prefill/decode	Instant prefill in legacy mode; explicit prefill budget and contention diagnostics in prefill-aware modes.
Resource limits	Active-sequence, token-budget, and KV-demand constraints as implemented by the configured policy/model.
Scheduling decisions	Policies observe simulator state and decide admission, ordering, and supported control actions.
Metrics	Computed from arrivals, completions, drops, deadlines, and per-step histories; undefined conditional metrics are excluded, not imputed.
Omitted hardware effects	No claim of absolute latency or throughput fidelity to physical GPUs; kernel, networking, allocator, and hardware-noise effects are abstracted except in the selected real-system validation.

active decoding request produces one output token per step. In the prefill-aware mode used by faithful serving-style policies, requests may first consume prefill budget before decoding begins; decoding requests produce one token per step; chunked-prefill policies may limit the prefill work admitted into a step; and prefill/decode contention and token-budget saturation are recorded as diagnostics. Admission-control and SLO-aware policies may decline otherwise waiting requests or hold decode work according to their policy semantics; SLO/deadline outcomes are then evaluated through the metric contract (§5.7).

Simulation terminates when all arrivals have been processed and no request remains waiting, active, migrating, or relocating, or when the configured post-arrival drain limit is reached. Requests left in the waiting or transfer states at termination are treated as dropped. Metrics are computed from completed requests, dropped requests, the full arrival population, and the per-step histories; no completion-conditioned metric is imputed when no request completes (§5.7). Given identical requests, configuration, policy, and operating region, the simulator consumes no randomness and produces bit-identical outputs; this was verified by a repeated-run determinism test. This determinism is what permits the paired design of §3.4: two policies' outcomes on the same window and region differ only because of the policy's own scheduling decisions, not because of independent sampling noise.

5.4 Verification, Validation, and Domain of Validity

The study separates five forms of evidence. First, implementation verification consists of simulator and policy unit tests, schema/structural validators for campaign outputs, the repeated-run determinism check, and policy-specific fidelity audits where a policy is labeled `FAITHFUL_EXTERNAL` (§5.2). Second, input/workload validity is supported by using independently released traces, preserving source-native provenance where fields exist, and characterizing the frozen workload corpus rather than assuming source interchangeability (§4). Third, experimental-design validity is provided by the preregistered freeze of policies, windows, operating regions, metrics, and analysis rules before comparative outcomes were generated, together with the paired design and outcome-independent policy-selection rule (§3). Fourth, physical-system validation is limited to the selected vLLM/GPU cases in §8: the goal is to test relative ordering and reversal direction for those cases, not to validate absolute simulated latencies or every cell in the matrix. Fifth, the domain of validity is therefore

the tested simulator, workload windows, policy panel, operating regions, metrics, and selected validation cases; no claim is made that the simulator is globally accurate for all LLM-serving systems or that every simulated reversal would transfer to physical hardware.

5.5 Discriminability Pilot (Historical)

Before the benchmark protocol described in the remainder of this section was frozen, a preregistered discriminability pilot (internally labeled Stage 0) executed a real, 1,080-cell matrix (three sources, three load regions, a six-policy PRIMARY subset) to check, before committing to the full protocol, that the panel would yield non-trivial scheduler differentiation under a fixed load protocol without post-hoc cherry-picking. **This is historical pilot evidence, not the confirmatory benchmark-campaign result**, and it never measured cross-source ranking portability – that question was and remains explicitly out of the pilot's scope. Its own five-criterion discriminability analyzer returned four passing criteria (77.8% of (source, window, load-region) triples non-tied, 86.7% of windows non-tied, no universal policy collapse, and 100% of non-tied cells showing meaningful metric variation) and one genuine failure: BurstGPT contributed only 14.3% of non-tied conditions, against a 15% floor, versus ∼42.9% each for the other two sources – a failing criterion, robust to the completion-conditioned-metric zero-completion convention used, that was reported at the time as a preregistered no-go on the pilot panel. A follow-up diagnostic traced this to a mechanism-level explanation: differentiation was strongly source-dependent, attributable to a near-total collapse of five of the six pilot policies into pairwise bit-identical outcomes on BurstGPT's structurally short-prompt, near-constant-output traffic – not to BurstGPT receiving weaker or fewer calibrated load conditions than the other sources. This motivated the benchmark protocol's larger, mechanism-diverse 13-policy panel and denser six-region load grid (§5.6), applied symmetrically to every source, never to BurstGPT alone. Detailed pilot mechanism analysis is reported in the reproducibility artifact (§11) rather than in the main text; it is not restated here as a ranking-portability finding.

5.6 Operating-Region Calibration

A FIFO-only calibration pass establishes six operating regions – `LOW` (load factor 0.5), `PRE_KNEE` (0.8), `KNEE` (1.0), `POST_KNEE` (1.1), `OVERLOAD` (1.2), and `HIGH_PRESSURE` (1.5) – across all 120 frozen windows (720 calibration cells total; 720 valid, deterministic, 0 failed/missing/ duplicated). We report this as calibration provenance, not a comparative scheduler result: FIFO is the calibration reference policy, and the calibration's role is to fix load-factor-to-region mappings before any other policy is executed against them, not to characterize FIFO's relative performance. As a calibration characteristic worth preserving, 301 of the 720 calibration-region records (93 BurstGPT, 101 Bailian/Qwen, 107 Azure) recorded zero completions under FIFO at that region's load factor; this is a property of the calibration design at those specific (window, region) combinations, the frozen calibration nonetheless passed its validity check, completion-conditioned metrics are simply undefined (not zero) in those records (§5.7), and no calibration redesign was performed after observing this count.

5.7 Metrics

Three metrics are always defined for every completed cell: completion fraction, weighted completion fraction, and Arrival-Normalized Weighted Goodput (ANWG, the primary metric). We formally define ANWG here because it is this benchmark's headline measure of system-level utility. For a set of arriving requests R, let $w_i \geq 1$ denote the priority weight assigned to request $i \in R$. Let $C \subseteq R$ be the subset of requests that successfully complete execution, and let $S \subseteq C$ be the subset of completed requests that also strictly meet their latency deadline (SLO). ANWG is defined as the priority-weighted sum of successfully attained deadlines divided by the total priority weight of all

arrivals:

$$\text{ANWG} = \frac{\sum_{i \in S} w_i}{\sum_{i \in R} w_i} \tag{1}$$

Because the denominator is the sum over all *arrivals* rather than completions, rejected, dropped, unfinished, and SLO-missed requests all receive zero numerator credit while remaining in the denominator penalty. ANWG is always defined (the denominator is zero only if a workload window has no arrivals, which is prevented by construction); therefore, undefined-metric exclusions for ANWG are always exactly zero. A second group of metrics is defined only conditional on at least one request completing: SLO-violation rate, weighted goodput, mean/p95/p99 latency, and request/token throughput are each `NaN` if and only if completion fraction is exactly zero for that cell. Mean/p95 time-to-first-token (TTFT) is subject to a stricter, separately checked precondition (at least one completed request recorded a first-token time), because TTFT can be undefined even when completion fraction is positive. No undefined conditional-metric value is ever imputed as 0 or 1, in either direction, and never after inspecting which policy or source it would affect. Where a policy's value is undefined for a given (window, region, metric) condition, that policy is excluded from the ranking-derived statistic (Kendall's tau-b, Spearman's rho, top-k overlap, pairwise reversal detection) for that specific condition and metric only – never scored as a loss or a tie, and never causing the whole condition to be dropped for policies that did complete. Every ranking-derived table reports its exclusion count (`n_conditions_excluded_for_undefined_metric`) alongside the statistic itself, since a source-concentrated exclusion rate is itself a reportable finding under RQ4, not a footnote.

5.8 Paired Design and Repetitions

Because the simulator is fully deterministic (§5.3), a single execution per (window, policy, region) cell is sufficient: each of the 9,360 configurations yields one scientific outcome, and its rep0/rep1 pair (18,720 executions in total) is a deterministic verification pass confirming bit-identical output, not an independent stochastic replicate. Ranking-uncertainty quantification (§6) instead comes from resampling over the independent workload windows themselves (block bootstrap), not from repeated stochastic runs of the same cell.

5.9 Mechanism Telemetry

Every executed cell additionally records a seven-field mechanism-activation telemetry summary: queue-depth (mean/max), batch-saturation (mean/max fraction of `max_active_sequences` in use), prefill/decode contention fraction, KV-occupancy (mean/max fraction of `max_kv_tokens` in use), admission-control activation count, preemption/reorder event count, and token-budget saturation fraction. Each field is always defined: a value of zero means either that the corresponding mechanism is genuinely absent for that policy (for example, preemption count is exactly zero for every policy that never issues a preempt/swap/migrate action, which is most of the panel) or that the underlying simulator mode does not implement a separate prefill/decode phase or token-budget model – not an instrumentation failure. This telemetry is collected for every cell but is not used to support any mechanism-level finding in this manuscript; RQ4's pre-specified telemetry-to-reversal association analysis is reported, with its sparsity and significance caveats, in §9.5.

5.10 Ranking-Uncertainty Quantification and Reversal Classification

Because the statistical unit of analysis is the workload window (§3.4), ranking-uncertainty is quantified by a whole-window block bootstrap: windows, not individual requests, are resampled with replacement (preserving each window's internal request sequence intact), 2,000 resamples are drawn per condition, and every reported confidence interval is a 95% percentile interval over

those resamples, using a deterministic, recorded seed so every interval is exactly reproducible from the frozen outcome matrix alone. This block structure exists because the requests within a window are not independent observations of scheduler behavior – they share one arrival process, one prompt/output-length mixture, and one operating region – so resampling at the request level would understate uncertainty; the window is the unit that is actually drawn independently by §4.3's sampling design, and it is therefore the unit that must be resampled.

Two further checks calibrate how the pairwise reversal statistics reported in §6.3 are interpreted, and it is important not to conflate them. Although a Friedman rank-sum omnibus test was originally specified in the protocol to test for global differences, it was never implemented; we disclose this as a protocol gap and proceed directly to the pairwise tests below. Separately, because the panel and the (source, region, metric) grid together generate many pairwise comparisons, every pairwise reversal test's two-sided bootstrap sign-test p-value – the intersection-union combination $\max(p_x, p_y)$ of the two compared conditions' sign-test p-values, read off the same resamples that back the ordinary CI rule below – is additionally checked against a Benjamini–Hochberg false-discovery-rate (FDR) correction at $q = 0.05$, applied per family, where a family is all pairwise policy comparisons within a single (metric, load-region) combination. This produces a separate, multiplicity-corrected `supported_after_fdr` flag on every record. In the frozen benchmark artifact, all 36 records that the five-class taxonomy below independently classifies as `SUPPORTED_PRACTICAL_REVERSAL` also satisfy `supported_after_fdr` – the FDR check removes no headline reversal in this dataset – but the two criteria are computed independently from each other and are not guaranteed to coincide in general.

A numerical sign change in a point estimate is not, by itself, a meaningful scheduler reversal from a systems standpoint: two policies can swap point-estimate order by a magnitude far smaller than the benchmark's own measurement noise, in which case treating it as a reversal would overstate what the benchmark actually supports. The 10% winning-margin threshold used throughout this test was fixed prospectively, before any reversal was observed, by reusing this project's own earlier discriminability-pilot criterion (`_relative_range_qualifies`, §5.5) rather than adopting a threshold chosen after seeing these results or asserted as a domain-standard cutoff; it is an operational convention for separating materially sized ordering changes from microscopic sign changes, not a claim that 10% is a universal practical-significance threshold for LLM serving, and every count labeled "practical" in this manuscript is conditional on it. Every pairwise (policy A, policy B) comparison at a given (source, region, metric) condition is therefore classified into exactly one of five preregistered classes, combining the practical-margin test with the *ordinary* (not FDR-adjusted) 95% percentile bootstrap-CI test defined above; the FDR check described above is a separate, additional multiplicity safeguard applied on top of this classification for headline reporting, not an input the classification itself depends on:

`UNDEFINED_UNESTIMABLE` At least one of the two policies' metric values is undefined for this condition (§5.7); no direction is estimated or claimed.

`STABLE_NO_SIGN_CHANGE` The two policies' relative order agrees across the compared conditions; no reversal occurred.

`MICROSCOPIC_SIGN_CHANGE` The point-estimate order reverses, but the winning margin fails to exceed 10% of the losing policy's value in *at least one* of the two compared conditions – this includes both the case where the margin is negligible on both sides and the asymmetric case where it exceeds 10% on one side but not the other. Either way the sign flip is reported here as numerically real but practically negligible or unreliable, never pooled with the classes below.

UNSUPPORTED_SIGN_CHANGE_WIDE_CI The point-estimate order reverses and the winning margin exceeds 10% of the losing policy's value in *both* compared conditions, but the *ordinary* (not FDR-adjusted) 95% bootstrap interval for the difference fails to exclude zero in *at least one* of the two conditions – the apparent reversal is not statistically distinguishable from bootstrap noise, at the preregistered confidence level, in at least one of the two conditions being compared.

SUPPORTED_PRACTICAL_REVERSAL The point-estimate order reverses *and* the winning margin exceeds 10% of the losing policy's value in both compared conditions *and* the *ordinary* (not FDR-adjusted) 95% bootstrap interval for the direction excludes zero in both conditions. Only this class is reported as a headline reversal in §6.3; the other four classes are always reported alongside it, never omitted as a residual category. Every record in this class additionally satisfies the separate FDR-corrected `supported_after_fdr` check described above in the frozen benchmark artifact.

These five classes are mutually exclusive and exhaustive by construction: `classify_pairwise_reversal` in the frozen analysis code evaluates undefined-value, sign-change, margin, and confidence-interval conditions as a strict if/elif/else chain with an unconditional final branch, so every possible (policy A, policy B, condition) input reaches exactly one class, including the asymmetric-margin and single-sided-wide-CI cases described above; no already-computed reversal record is at risk of being silently dropped or misclassified by a gap in this taxonomy.

5.11 Temporal Analysis Design

Temporal portability is evaluated separately per source, using each source's own native chronology rather than a single shared calendar rule, because the three sources differ in what temporal structure they actually expose (§4.1). BurstGPT windows are assigned to EARLY/MIDDLE/ LATE chronology terciles by deterministic relative order within the trace (§4.3; 60/30/30 windows respectively), and a secondary EARLY/LATE bisect split is run as a robustness check on whether the tercile boundary choice itself drives any temporal finding (§5.12). Azure 2024's temporal split instead uses the trace's own absolute calendar timestamp, frozen at Unix epoch `1715731200.0` (2024-05-15T00:00:00Z), splitting the trace into before/after halves at that boundary; this is a genuine calendar split, not a relative-order tercile, because Azure's release exposes an absolute timestamp field that BurstGPT's schema does not (§4.2). Bailian/Qwen's release does not expose a verified absolute timestamp field usable for a calendar split; its temporal analysis is therefore restricted to a relative-order comparison only, explicitly labeled `RELATIVE_CHRONOLOGY_ONLY` in every table it appears in, and is never presented alongside BurstGPT's tercile result or Azure's calendar-split result as though the three used an equivalent temporal definition (§6.4). No cross-source temporal number is computed by pooling these three distinct temporal definitions into one figure.

5.12 Robustness Analyses

Every headline ranking-portability result (§6) is pre-specified to be checked against a fixed set of robustness analyses: the 11-policy high-fidelity subset (PRIMARY panel excluding both `STYLE_APPROXIMATION` policies), leave-one-source-out re-estimation, window-size sensitivity, an alternate completion-conditioned-metric convention, the full six-region load grid, a temporal-split sensitivity check (§5.11's BurstGPT bisect), and mechanism-family-exclusion checks. Six of these seven were implemented and are reported in §6.5; the seventh, `METRIC_DEFINITION_SENSITIVITY`, was never implemented and is disclosed there as an open item rather than a result.

5.13 SLO-Definition Sensitivity: A Disclosed Protocol Gap

An SLO-definition sensitivity check – re-running the benchmark's deadline-dependent policies under an alternative SLO-synthesis rule to confirm that any headline reversal is not an artifact of the one synthesis choice used to fabricate `slo_deadline` where a source does not natively carry it (§4.2) – was planned as part of this protocol, but no alternative SLO-synthesis rule was ever actually frozen before or during the benchmark campaign. We disclose this as a matter of methodological discipline rather than as a result: this robustness component is *not* claimed as preregistered evidence anywhere in this manuscript. It was instead run afterward as an explicitly labeled post-campaign extension, with its own frozen protocol, after the campaign's headline results were already known; its results are reported in §6.7 and §10.

6 Scheduler Ranking Portability

The results below are computed deterministically from the frozen, independently validated outcome matrix of 9,360 configurations (§5.8), using the prespecified analysis pipeline (full code and data provenance in §11); no number below was entered by hand. The PRIMARY headline metric throughout is arrival-normalized weighted goodput (ANWG, §5.7); other metrics are reported as secondary/supporting evidence only. Two items from the original result contract are not part of this primary analysis: per-source *discriminability* (fraction of non-tied conditions), which remains an open item rather than populated with placeholder or approximated numbers; and cross-*metric* rank agreement, reported instead in §6.6 as an explicitly labeled post-campaign extension that reuses this analysis's own statistical primitives without modifying or re-running the underlying matrix.

The section separates three distinct notions that should not be conflated: portability of simulated rankings across workload sources and operating regions (§6.1–§6.3); portability of rankings across evaluation metrics (§6.6); and simulator-to-physical fidelity, which is evaluated only in the selected real-system cases of §8.

6.1 Cross-Source Rankings

Across the 3 source pairs × 6 operating regions (18 conditions), Kendall's tau-b for the PRIMARY metric ranges from 0.55 to 1.00, with a per-region mean between 0.69 and 0.77 (Table 2, Figure 1). Every one of the 18 conditions has a positive, non-trivial tau-b: comparative scheduler rankings are never uncorrelated or inverted *in aggregate* between any two sources at any region. Portability is, however, visibly uneven across source *pairs* rather than uniform: Azure-2024 and Bailian/Qwen agree very strongly with each other at every region ($\tau = 0.89$–1.00), while every comparison involving BurstGPT is substantially lower ($\tau = 0.55$–0.71) – a gap of roughly 0.2–0.4 in Kendall's tau-b that holds at all six regions, not just one. Top-1 agreement (the single best-performing PRIMARY-panel policy) and top-3 agreement follow the same source-pair pattern in the underlying evaluation data; we do not tabulate top-1/top-3 separately in the main text to avoid restating the same qualitative pattern three times, but the full table is part of the public artifact release (§11).

The min/max range in Table 2 makes clear that portability is not homogeneous even within a region: the range at LOW (0.55–0.92) is nearly as wide as at any other region, and the range is driven entirely by which source pair is compared, not by which region is compared (§6.2 tests region dependence explicitly). No region shows uniformly low or uniformly high agreement across all three source pairs simultaneously.

Table 2. Primary-metric (ANWG) Kendall's tau-b, mean across the three source pairs, per operating region. ANWG is always defined; undefined-metric exclusions are 0.

	LOW	PRE_KNEE	KNEE	POST_KNEE	OVERLOAD	HIGH_PRESSURE
mean τ	0.69	0.77	0.74	0.74	0.72	0.75
min τ	0.55	0.70	0.60	0.61	0.58	0.62
max τ	0.92	0.89	0.96	0.96	0.96	1.00

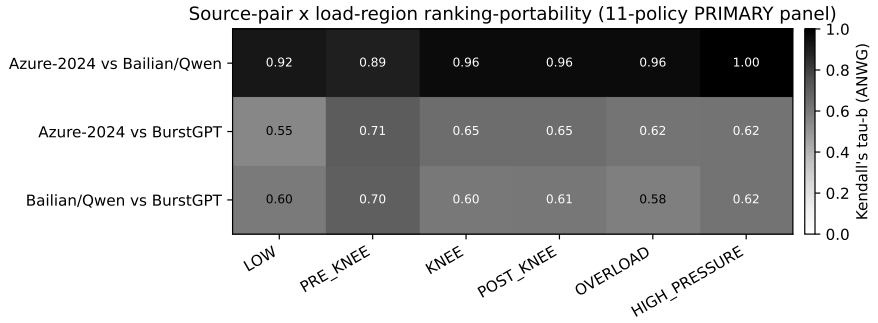


Fig. 1. Source-pair × load-region ranking-portability: Kendall's tau-b between each pair of sources' PRIMARY-panel (11-policy) rankings on the ANWG metric, at each of the six frozen operating regions (§5.6). Computed from the frozen ranking correlations artifact.

6.2 Load Conditioning

Load-level dependence is a secondary robustness/sensitivity analysis (§3.1), reported here as a direct read of the same §6.1 table rather than a separate headline claim of cross-source rank instability by itself. Secondary ranking statistics (Spearman's ρ and top-1/top-3 overlap) exhibit an identical quantitative pattern and are omitted from the main tables for concision; their full values are preserved in the released analysis outputs. Table 2's per-region mean tau-b is close to flat across the six regions (0.69 at LOW, rising to 0.72–0.77 elsewhere, no monotonic trend with load factor); a leave-one-region-out comparison between the full six-region grid and the four-region `LOW/PRE_KNEE/KNEE/ OVERLOAD` subset used by `LOAD_CALIBRATION_SENSITIVITY` (§5.12) changes the mean tau-b by less than 0.01 (0.728 on the four-region subset versus 0.733 on the full six-region grid) – the headline cross-source portability conclusion is not sensitive to the choice of four-region versus six-region calibration grid. Load regime does, however, materially affect *where practically supported reversals occur* (§6.3): reversal counts are concentrated at PRE_KNEE and above, essentially absent at LOW.

6.3 Pairwise Reversals

Of the 990 (policy pair) × (source pair, region) PRIMARY-metric records (55 policy pairs among the 11 PRIMARY policies × 18 conditions), the five-class breakdown (§5.10) is: 906 `STABLE_NO_SIGN_CHANGE` (91.5%), 48 `MICROSCOPIC_SIGN_CHANGE` (4.8%), 36 `SUPPORTED_PRACTICAL_REVERSAL` (3.6%), and zero records in either `UNDEFINED_UNESTIMABLE` or `UNSUPPORTED_SIGN_CHANGE_WIDE_CI`. The large majority of pairwise comparisons are stable; a small but non-trivial minority (about 1 in 28) are both practically and statistically supported reversals – never a majority, and never absent.

That zero records fall into UNSUPPORTED_SIGN_CHANGE_WIDE_CI means that, at this sample size, every sign change with a practically meaningful margin also had a statistically supported direction, and vice versa – on this data the practical-margin and statistical-support criteria never disagreed with each other; this is a description of the observed record set, not independent confirmation that the two criteria would always agree.

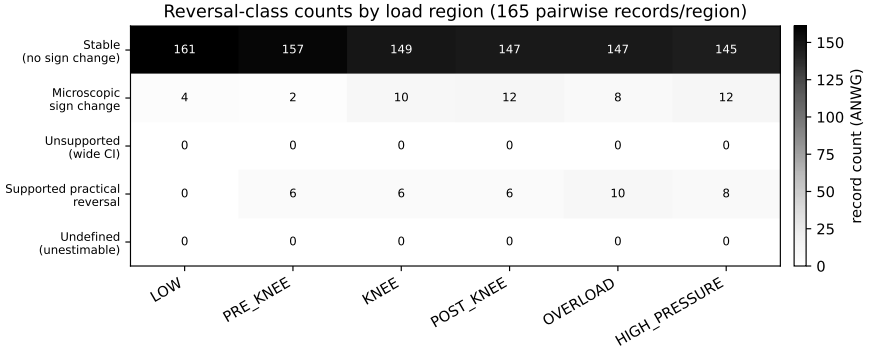


Fig. 2. Reversal-class counts by operating region, PRIMARY metric (165 pairwise records per region: 55 policy pairs × 3 source pairs). Computed from the frozen pairwise reversals artifact.

Reversal counts are load-dependent: zero SUPPORTED_PRACTICAL_REVERSAL records at LOW, rising to 6 at each of PRE_KNEE, KNEE, and POST_KNEE, 10 at OVERLOAD, and 8 at HIGH_PRESSURE (Figure 2). Every one of the 36 PRIMARY-metric supported reversals that we manually inspected in the underlying record set involves the same two policies, slai_faithful and vllm_faithful, and every one is a *cross-source* reversal in which one side of the comparison is a BurstGPT condition – consistent with §6.1's finding that BurstGPT is the source pair member most associated with lower agreement. The single largest-effect-size supported reversal (operationalized as the smaller-magnitude of the two conditions' practical margins, the binding constraint under the frozen practical-margin rule, §5.10) is slai_faithful vs. vllm_faithful at HIGH_PRESSURE, comparing Azure-2024 (or, at an effectively tied effect size, Bailian/Qwen) against BurstGPT – this is the case selected for real-system validation under the frozen selection rule (§8).

A representative stable comparison, used as a null-result control alongside the reversal case in the real-system validation design (§8), is the source pair with both the highest tau-b and (among the highest-tau conditions) the narrowest bootstrap interval: Azure-2024 vs. Bailian/Qwen at HIGH_PRESSURE ($\tau = 1.00$, 95% CI $[0.90, 1.00]$).

6.4 Temporal Portability

Following the frozen temporal design (§5.11), four temporal record types are reported separately, each under its source's own chronology, and never merged into a single "temporal/domain OOD" number: BurstGPT's EARLY/MIDDLE/LATE tercile split (primary) and EARLY/LATE bisect (sensitivity check on the tercile boundary), Azure-2024's absolute calendar split, and Bailian/Qwen's RELATIVE_CHRONOLOGY_ONLY comparison, for which no calendar-based claim is made. All values are keyed identically to the source temporal robustness artifact. The canonical results reveal high rank stability across time: BurstGPT's tercile split shows strong agreement between EARLY and LATE chunks across non-LOW load regions ($\tau_b \in [0.81, 1.0]$), while the BISECT sensitivity check yields similarly stable correlations ($\tau_b \in [0.72, 0.80]$). Azure's calendar split produces near-perfect

correlation ($\tau_b \in [0.90, 1.0]$), and Bailian's relative-chronology split likewise exhibits minimal rank change ($\tau_b \in [0.94, 1.0]$).

Although the frozen protocol for this research question specified an analysis of temporal, provider, and domain shifts, a standalone domain-shift component was not separately operationalized in the final artifacts. The evidence reported here addresses the temporal component directly. The frozen cross-source comparisons (§6.1) include the one genuine provider boundary in this corpus (Bailian/Qwen on Alibaba Cloud versus the two Microsoft-infrastructure sources, §4.1), but the design does not isolate a causal provider effect from ordinary cross-source variation – BurstGPT-vs-Azure is a cross-source, not cross-provider, comparison; domain shift, like the provider component, is addressed only implicitly through these same cross-source comparisons and was not separately operationalized.

6.5 Robustness

Of the seven preregistered robustness families (§5.12), six were computed (five in the ranking-correlation, temporal-robustness, and sample-complexity outputs below, and a sixth, `LEAVE_ONE_POLICY_FAMILY_OUT`, in the telemetry-explanation analysis of §9.5). The seventh, `METRIC_DEFINITION_SENSITIVITY`, was never implemented; we disclose this as a protocol gap rather than perform a post hoc analysis after the results were already known.

- **`PRIMARY_ONLY`.** Identical to the headline result by construction: the ranking correlations artifact already restricts every comparison to the 11-policy PRIMARY panel (§6.1), never mixing in the two `STYLE_APPROXIMATION` robustness policies.
- **`LEAVE_ONE_SOURCE_OUT`.** Excluding BurstGPT leaves only the Azure-2024/Bailian-Qwen pair, whose mean tau-b is 0.948 – close to the ceiling. Excluding Azure-2024 or Bailian/Qwen each leaves a BurstGPT-involving pair, with mean tau-b 0.633 and 0.618 respectively. This is the same asymmetry as §6.1, now stated per excluded source: the headline portability figure is high specifically because two of the three sources agree strongly, and is pulled down specifically by BurstGPT, not uniformly by all three sources.
- **`LOAD_CALIBRATION_SENSITIVITY`.** Reported in §6.2: mean tau-b changes by less than 0.01 between the four- and six-region grids.
- **`TEMPORAL_SPLIT_SENSITIVITY`.** The BurstGPT bisect-vs-tercile comparison from §6.4; both split definitions are reported side by side in the released table rather than collapsed into one number.
- **`WINDOW_SIZE_SENSITIVITY`.** Despite its name, this item is the window-*count* subsampling ladder of §7, not a requests-per-window sweep (§4.8), and is not separately recomputed here.
- **`LEAVE_ONE_POLICY_FAMILY_OUT`.** Excluding any single one of the ten frozen mechanism families changes the headline mean tau-b only modestly, from 0.707 (excluding the classical arrival-order family) to 0.792 (excluding the SLO-aware family), against the full-panel baseline of 0.733. No single mechanism family's removal collapses or inflates the headline portability figure – the source-pair-driven pattern of §6.1 is not an artifact of any one policy family dominating the panel.

6.6 Cross-Metric Ranking Portability

§6.3 and §6.5 ask whether a policy ranking under one fixed metric (ANWG) is portable across sources, regions, and windows. A different question – whether a ranking is portable across *metrics*, for the same source and region – is addressed here as an explicitly labeled post-campaign extension that reuses the primary analysis's own statistical methods without modifying, re-running, or re-deriving the underlying outcome matrix. Across all 3 sources × 6 regions × 55 metric pairs

among the study's tracked outcome metrics (990 conditions), rank correlation between metrics is on average moderate (median Kendall's $\tau_b = 0.419$, range $[-0.506, 1.000]$; 202/990, 20%, of conditions have *negative* τ_b), and the single top-ranked policy agrees between the two metrics in only 316/990 (31.9%) of conditions (disagreeing in the remaining 68.1%). At the level of individual pairwise policy comparisons (54,450 total, one per (metric pair, condition, policy pair), generated from the 55 unique pairs among the 11 eligible tracked metrics: ANWG, completion fraction, weighted completion fraction, SLO-violation rate, weighted goodput, mean latency, p95 latency, request throughput, token throughput, mean TTFT, and p95 TTFT; p99 latency was ineligible and excluded), the same five-class taxonomy as §6.3 (ordinary, not FDR-adjusted, bootstrap-CI classification; §5.10) finds 3,244 (6.0%) show a statistically supported practical disagreement in which policy is preferred depending on which metric is used, versus 45,800 (84.1%) in the same order, 5,205 (9.6%) a sign change too small to be practically meaningful, and 201 (0.4%) an unsupported sign change with a wide confidence interval; no comparison in this policy panel was undefined or unestimable. Unlike §6.3's primary reversal check, the separate BH-FDR correction removes some of these cross-metric disagreements: of the 3,244 raw-CI-supported records, 3,207 (98.9%) also survive the family-wise BH-FDR check, while 37 (1.1%) do not, meaning the multiplicity-corrected count of cross-metric disagreements is 3,207 rather than 3,244 – a genuine, disclosed sensitivity to the multiplicity correction, not an error in either figure. Scheduler-ranking portability is therefore not only source- and region-dependent (§6); which metric is used to define "better" materially changes the ranking in a non-trivial minority of conditions, and the median condition shows no default expectation that the two metrics' top choices should agree.

6.7 SLO-Definition Sensitivity

As disclosed in §5.13, this check was never part of the preregistered protocol; we report it here as the post-campaign extension it was actually run as: three alternative SLO-synthesis multipliers (`tight_10x`, `primary_20x` – this study's own convention – and `loose_40x`) applied across the 5 load regions that contain at least one PRIMARY-metric supported reversal (excluding `LOW`, which accounts for 0 of the 36 supported reversals and so has nothing for a sensitivity check to stress-test), evaluating 11 policies × 120 total windows × 5 regions × 3 variants (19,800 cells; 0 failed), with every seed held identical to §6.

Ranking robustness. Across the 30 (source, region, variant) conditions compared against the study's primary SLO convention, rank correlation stays high under either alternative (median Kendall's $\tau_b = 0.872$, range $[0.533, 1.000]$); the single top-ranked policy nonetheless changes under an alternative SLO definition in 13/30 (43%) of conditions.

Reversal persistence. We check the 36 PRIMARY-metric supported reversals from §6.3 against the alternative definitions. Because there are 3 SLO variants (tight, primary, loose), there are $36 \times 3 = 108$ mathematical comparisons in the artifact's persistence matrix, of which 36 are identity checks against the primary definition itself (all 36 trivially persist). Restricting focus to the 72 scientifically meaningful non-trivial comparisons against the alternative definitions: 52/72 (72%) persist, 6/72 (8%) change direction, 10/72 (14%) remain the same direction but fall below the practical-margin threshold under the alternative definition, and 4/72 (6%) become statistically indistinguishable from zero. No headline reversal from §6.3 is contradicted outright, but a non-trivial minority (20/72, 28%) are sensitive to the SLO-definition choice – disclosed here as a caveat on those specific reversals, not grounds to discard them.

6.8 Synthetic-to-Real Ranking Transfer (Pilot, Not Headline Evidence)

RQ3 (§3) is preregistered as a SECONDARY question, never a headline result contract item. We report only an engineering- validation pilot of the synthetic-to-real transfer pipeline: 176/176 pilot

cells executed and 24/24 analysis records generated, 0 schema-validation or telemetry failures, across all 4 synthetic stress families (`burst_arrival`, `decode_length_skew`, `heavy_tail_service`, `priority_slo_heterogeneity`), both selected load regions, and all 3 real sources. By design, the pilot uses only 2 synthesis seeds per family, too few for the frozen block-bootstrap transfer statistic to be defined in most conditions: Kendall's τ_b is undefined (`null`, insufficient sign pairs) in 21/24 (88%) pilot conditions. **We therefore draw no synthetic-to-real transfer conclusion, positive or negative, from this pilot**; it establishes only that the pipeline executes correctly end-to-end. A full-scale extension (targeted, not exhaustive) with sufficient seeds per family to define the transfer statistic is left to future work, consistent with RQ3's SECONDARY, non-headline status in this study's frozen contract.

7 Benchmark Sample Complexity

7.1 Motivation

Every scheduler-ranking result in this manuscript is itself derived from a finite window budget – 40 windows per source, 120 total (§4.3). A researcher designing a future simulation experiment for scheduler comparison faces a practical, resource-bound version of the same question this benchmark studies at the source/region/metric level: *how much independent workload evidence is required before a comparative ranking obtained from a simulation experiment becomes stable?* Equivalently, if only n workload windows can be afforded, how reliably would the resulting scheduler-ordering conclusion reproduce the conclusion obtained from the full window collection? This question is a simulation experimental-design parameter, not a purely statistical afterthought: it bounds the experimental cost of a trustworthy scheduler comparison, it affects how much confidence a reader should place in a paper that evaluates on a handful of trace windows, and it is a precondition for treating this benchmark's own frozen 120-window corpus (rather than some larger or smaller corpus) as an adequate evidentiary base for the results in §6.

7.2 Design

The 0.9 recovery probability used throughout this section is a pre-registered benchmark reporting target, fixed before any sample-complexity output was computed: it provides one interpretable, high-recovery criterion for comparing sources and metrics against each other, and is not asserted to be a universal statistical-significance level or a field-standard adequacy cutoff for scheduler benchmarking.

For the PRIMARY metric, recovery of the exact $n = 40$ reference ranking is slow and strongly source-dependent (Figure 3, Table 3): BurstGPT never reaches the preregistered 0.9 exact-order recovery threshold below the full window budget (0.2% at $n = 5$, rising only to 56.6% at $n = 30$), Bailian/Qwen similarly does not cross 0.9 before the $n = 40$ ceiling (64.0% at $n = 30$), while Azure-2024 crosses the threshold already at $n = 30$ (98.8%). None of the three sources reaches 0.9 exact-order recovery at $n \in \{5, 10, 20\}$. Because $n = 40$ is the size of the full reference pool itself, exact recovery at $n = 40$ is 1.0 by construction (sampling all 40 windows without replacement reproduces the reference exactly) and is not an informative data point on its own; the informative content is in how slowly each source's curve rises toward that trivial ceiling.

Top-1 recovery (does the resampled subset identify the single best-performing PRIMARY-panel policy correctly) is a materially easier question than exact-order recovery for two of the three sources: Azure-2024 and Bailian/Qwen both exceed 0.9 top-1 recovery already at $n = 5$ (96.6% and essentially 100% respectively), even though their *exact*-order recovery at $n = 5$ is only 35.0% and 14.6%. For these two sources, identifying the single best policy requires far less evidence than reconstructing the complete 11-policy ordering. BurstGPT shows no such gap: its top-1 recovery is

comparably slow to its exact-order recovery (13.0% at $n = 5$, 74.2% at $n = 30$), remaining below 0.9 until the trivial $n = 40$ ceiling – consistent with §6.1's finding that BurstGPT is the source whose relative ordering is hardest to pin down. Comparing a concentrated (single-source-heavy) against a distributed (evenly-spread-across-sources) window budget of equal total size, the distributed budget achieves higher mean Kendall's tau-b against the reference ranking (0.986) than the concentrated budget (0.940) for the primary metric – spreading a fixed evaluation budget across sources recovers the reference ranking more reliably than concentrating it in one source.

Table 3. Smallest n at which PRIMARY-metric recovery probability first exceeds 0.9 ($n \in \{5, 10, 20, 30, 40\}$). Recovery at $n = 40$ is 1.0 by construction as the entire 40-window reference pool is sampled without replacement.

Source	first n, exact-order ≥ 0.9	first n, top-1 ≥ 0.9
Azure-2024	30	5
Bailian/Qwen	40 (not reached below 40)	5
BurstGPT	40 (not reached below 40)	40 (not reached below 40)

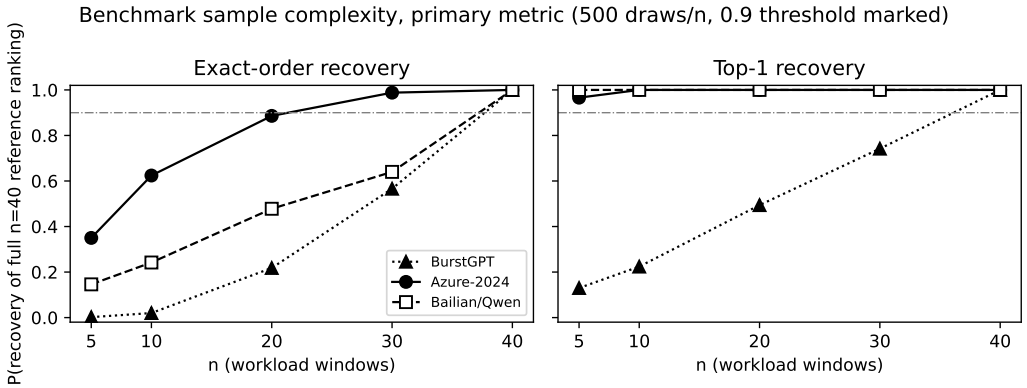


Fig. 3. Benchmark sample-complexity recovery curves, PRIMARY metric, 500 draws per n, sampling without replacement. Left: exact-order recovery of the full $n = 40$ reference ranking. Right: top-1 recovery. The 0.9 threshold is marked. Computed from the frozen sample complexity artifact.

This analysis directly answers RQ5 (§3.1) and depends on the full primary outcome matrix (§6); it is not run on the earlier, smaller discriminability pilot (§5.5).

8 Real-System Model-Fidelity Validation

8.1 Objective

This analysis addresses RQ6 (§3.1) as a selected model-fidelity and domain-of-validity check. We do not claim that simulated absolute latency or throughput values match real-hardware values; simulator latency is not assumed to transfer directly to real-engine latency, nor is the simulator's own load-calibration reference λ_{ref} (§5.6) expected to transfer to a physical engine's saturation point. The sole objective of this validation is to test agreement in *relative policy ordering* and *reversal direction* between the simulator and a real vLLM deployment on a targeted set of validation cases. Agreement would support the selected simulated comparison; a disagreement would instead identify a boundary on how that simulated comparison should be interpreted.

8.2 Experimental Design

The real-system validation follows a preregistered, window-resampled evaluation design structured to mirror the simulated campaign without requiring exhaustive duplication. It is deliberately not a full physical re-execution of the 13-policy, six-region simulation matrix. The evaluation protocol is defined as follows:

- **Inferential Unit:** The statistical unit of analysis is the workload window, preserving the methodology established in §5.
- **Scale and Execution:** The design evaluates 40 independent workload windows per source, each containing 200 requests. Exactly one scientific execution is performed per (policy, source, window) cell, yielding 240 total scientific tasks (2 policies $\times$ 3 sources $\times$ 40 windows).
- **Uncertainty Quantification:** Uncertainty is quantified via window-level bootstrap resampling ($\geq$ 2,000 resamples) over the 40 independent windows per source.
- **Operating Regions:** Workloads are calibrated against the real engine's observed saturation point to define load regions independently of the simulator's calibration.
- **Software and Hardware:** Every execution serves `Qwen/Qwen2.5-0.5B-Instruct` on `vllm==0.27.1` (`torch==2.13.0`), one instance per task on a single NVIDIA A100-SXM4-80GB GPU (Slurm resource label `gres=gpu:a100_10g`, unchanged from the validated load-calibration campaign). This label is a Slurm resource tag, not evidence of a fractional or MIG GPU partition; the allocated device is a full 80 GB A100. Tokenizer revision, model dtype, tensor-parallel degree, and maximum model length were not separately recorded in the frozen execution manifest; rather than invent values, we do not report them here.
- **Ordering and Provenance:** Each (policy, source, window) cell is executed exactly once; with one execution per cell there is no repeated-measures ordering to alternate. Cells are assigned deterministically to Slurm array indices by a fixed (source, window, policy) sort, with execution concurrency limited to four simultaneous array tasks. Every execution records its exact git commit, hardware environment (per-task GPU query), software versions, and workload/calibration-manifest hash identity to ensure reproducibility.

8.3 Case Selection

Validation cases were selected mechanically from the validated results presented in §6.3, applying a preregistered rule that prefers the pairwise reversal with the largest effect size and a representative stable-ordering control.

- **Reversal Case:** `slai_faithful` versus `vllm_faithful`, comparing the Azure and BurstGPT sources at high load. This case exhibited the largest-effect-size statistically supported practical reversal in simulation (a simulator difference of +0.520 for Azure favoring `slai_faithful`, and −0.348 for BurstGPT favoring `vllm_faithful`).
- **Stable Control:** Azure versus Bailian/Qwen at high load, chosen for exhibiting perfect rank agreement in simulation.

Evaluating this selection on physical hardware requires mapping the simulated mechanisms to their real-engine equivalents. While `vllm_faithful` maps directly to native vLLM execution paths, the SLO-aware SLAI/RAD mechanism requires a custom vLLM scheduling plugin to realize its semantics on real hardware.

8.4 Results

The physical-hardware load-calibration prerequisite for these validation cases has been executed (120/120 terminal outcomes: 43 converged, 77 lower-bound-already-violating), establishing the necessary injection rates for the deployment environment. All 240 scientific evaluation tasks (2

Table 4. Real-vLLM RQ6 outcomes against the simulator-selected cases (HIGH_PRESSURE, 40 paired windows/source, 95% window-level bootstrap CI, 2,000 resamples). Effect is slai_faithful−vllm_faithful ANWG.

Source	Sim. winner	Effect	95% CI	Real winner	Agreement
Azure (rev.)	SLAI (+0.520)	−0.024	[−0.027, −0.020]	vLLM	Disagrees
BurstGPT (rev.)	vLLM (−0.348)	−0.200	[−0.264, −0.144]	vLLM	Agrees
Bailian (ctrl.)	n/a*	−0.026	[−0.030, −0.022]	vLLM	Consistent

*The simulator's stable-control quantity ($\tau_b = 1.00$) is an 11-policy full-ranking correlation, not a two-policy slai_faithful-vs-vllm_faithful winner; no directly comparable simulator winner is defined for this row. The relevant real-system check is instead whether the two-policy ordering agrees between the Azure and Bailian/Qwen conditions, which it does (both favor vllm_faithful).

policies × 3 sources × 40 windows) subsequently completed on the software/hardware environment above (§8.2), with zero failed, timed-out, or partial cells.

Table 4 reports the slai_faithful-minus-vllm_faithful arrival-normalized-weighted-goodput (ANWG) effect for each source, using the same window-level paired bootstrap as §5.10 (2,000 resamples over the 40 independent windows/source, 95% CI). All three effects are statistically supported (CI excludes zero) and favor vllm_faithful.

The simulator-predicted Azure/BurstGPT reversal did not reproduce: vllm_faithful won both conditions on real hardware, so the predicted sign flip between Azure and BurstGPT was not observed. BurstGPT's real winner matches the simulator's selection; Azure's does not – the physical Azure comparison changed from a simulated slai_faithful advantage to a small but statistically supported vllm_faithful advantage. This negative validation result is scientifically informative: it identifies a fidelity boundary for the selected simulator-predicted major reversal and prevents that simulated reversal from being read as a hardware-confirmed result. In contrast, the Azure/Bailian-Qwen stable-control comparison retained a consistent vllm_faithful- favoring ordering across both real-system conditions (effect magnitudes −0.0235 and −0.0256), reproducing the qualitative stability that comparison was selected to test, independent of which specific policy wins.

Statistical support is not the same as practical magnitude. Read against this study's 10% practical-margin convention (§5.10), the three real-system effects vary substantially once expressed relative to the losing policy's own mean ANWG (frozen descriptive aggregates, docs/RQ6_PUBLIC_RESULT_PROVENANCE.md): Azure's margin is $(0.7700 - 0.7465)/0.7465 \approx 3.1\%$ and Bailian/Qwen's is $(0.84525 - 0.819625)/0.819625 \approx 3.1\%$ – both statistically supported but below the 10% convention used elsewhere in this study to separate practically meaningful from microscopic effects. BurstGPT's margin, $(0.316625 - 0.116375)/0.116375 \approx 172\%$, substantially exceeds it. This is a descriptive comparison against an existing convention, not a new inferential analysis: §5.10's five-class taxonomy is defined for a pairwise comparison across two conditions of the same source and is not applied here to a single physical-source row.

This selected-case result exposes a simulator-fidelity boundary rather than establishing a general finding: the Azure/BurstGPT reversal that the simulator identified as this study's largest-effect-size case did not transfer to physical execution, while the paired stable-control comparison retained its qualitative ordering (§10 scopes what this design does and does not establish; §9 interprets the two outcomes together). The appropriate interpretation is therefore not that simulation should be discarded, but that simulation-derived comparative rankings require an explicit model-validity context.

9 Discussion

The result (§6) is neither cleanly "portable" nor cleanly "fragile" – it is source-pair-dependent in a specific, mechanistically legible way, and that dependence itself is the finding. For simulation-based scheduler studies, the implication is that a comparative conclusion should be reported together with the workload, operating-region, metric, and model context that produced it.

9.1 Portability Is Real but Source-Pair-Dependent, Not Uniform

Comparative scheduler rankings are never uncorrelated between any two sources at any region (§6.1): a scheduler comparison performed on one of this study's three sources retains positive aggregate ranking agreement with the others. At the same time, portability is far from uniform, and the non-uniformity is legible rather than diffuse: Azure-2024 and Bailian/Qwen agree almost perfectly at every operating region, while every comparison involving BurstGPT is consistently lower (§6.1, §6.5). The practical implication is neither "a small, well-chosen evaluation footprint always suffices" nor "every scheduler comparison must be re-verified on every workload": it is that *which* second source a comparison is checked against matters more than *whether* a second source is checked at all. A comparison validated only on Azure-2024-like traffic shows strong agreement with Bailian/Qwen-like traffic under this study's panel and metric, but the same transfer to BurstGPT-like traffic is measurably less reliable, and that reliability gap holds at every load level tested, not only under stress.

9.2 Practically Supported Reversals Are Real, Uncommon, and Load-Concentrated

Practically and statistically supported reversals are a small minority of PRIMARY-metric pairwise comparisons (§6.3), but not zero, and not dismissible as measurement noise: the same bootstrap procedure classifies the large majority of comparisons as stable. Every supported reversal we examined is the same policy pair and involves BurstGPT on one side, and reversals are not spread evenly across the operating envelope; they concentrate at and above `PRE_KNEE`. A scheduler comparison performed only at low load, even on a source pair that is otherwise highly portable, would have missed every reversal this benchmark found. This motivates evaluating scheduler comparisons across an operating curve, not at a single convenient load point, as this benchmark's design already does (§5.6).

9.3 Implications for Simulation Experimental Design

Two implications follow directly and are not deployment recommendations about which scheduler to use, only about how to design simulation studies that evaluate schedulers credibly. First, a comparison checked against only one additional workload source cannot be assumed representative of "other workloads" in general – which specific source is used as the check matters, and BurstGPT-like traffic in particular is where this study's evidence concentrates its cross-source disagreement. Second, benchmark sample complexity is itself simulation-design evidence: §7's finding that exact-order recovery requires the full window budget for two of three sources, while top-1 recovery is achievable from very few windows for those same two sources, means a researcher who only needs to know "which scheduler is best" faces a different, much smaller evidence requirement than one who needs the complete ordering – a distinction a single point-estimate benchmark result cannot convey on its own.

9.4 Portability Also Depends on Which Metric and Which SLO Definition Is Used

§6.1–§6.3 establish that a ranking's portability depends on which *workload source* it is checked against. §6.6 and §6.7 show that portability depends, independently, on which *metric* and which

SLO definition is used to compute the ranking in the first place – two further conditioning variables, not a repetition of the source-pair finding above.

The cross-metric result is the more consequential of the two. Rank correspondence across metric pairs is only moderate (median $\tau_b = 0.419$) – well below the agreement between the two most portable *sources* – and metrics sometimes rank policies in opposite order rather than merely disagreeing in degree. Across the evaluated metric pairs within source-region conditions, the top-ranked policy differs more often than it agrees (§6.6). This is a cross-metric ranking-portability result, not evidence of a simulator-model failure. It does not mean metrics are arbitrary or that any one choice is as good as another – ANWG, completion fraction, and the latency- and SLO-based metrics measure genuinely different deployment objectives (§5.7), and a practitioner optimizing for tail latency has no reason to expect the same ranking as one optimizing for weighted goodput. The finding is instead that *which* objective a benchmark reports on is as much a part of the ranking-portability question as which workload it was measured on, and a scheduler comparison that reports only its primary metric is silent on whether that comparison would hold under a deployment's actual objective.

The SLO-definition result is distinct from metric sensitivity and comparatively reassuring, but not a clean pass: ranking robustness under alternative SLO-synthesis conventions is closer to the cross-source ceiling than to the cross-metric result, yet the top-ranked policy and a non-trivial minority of specific reversals do change with the SLO definition (§6.7). The practical reading is narrower than either "robust" or "fragile": a scheduler conclusion can remain broadly ordered under a nearby SLO definition even while its winning policy, or a specific reversal a reader cares about, does not.

Taken together with §6.1's source-pair finding and §6.3's load-concentration finding, this study's central conclusion is that a comparative scheduler claim is conditional on at least four axes – workload source, operating region, evaluation metric, and (for deadline-dependent policies) SLO definition – and reporting a ranking without stating which values of each axis produced it risks the ranking being read as more general than the evidence supports.

9.5 Explanatory Telemetry: A Preliminary, Heavily Qualified Read

The frozen, pre-specified explanatory model (§5.9) associates the five workload descriptors with reversal sites at each (policy pair, condition) cell, but converges with a usable sample size (≥ 20 observations) for only 42 of 990 cells – most reversal sites are too sparse for this per-cell model to fit reliably, and the frozen output records point coefficients only, with no standard errors or significance test. Any reading of this evidence must therefore be provisional. Within that small, non-significance-tested set, `prompt_tokens_cv` has a negative coefficient in 39 of 42 cells – the most consistent single direction among the five descriptors – which is at least *consistent with* (not confirmatory of) prompt-length variability being associated with where reversals occur, echoing §4.7's finding that prompt-length structure is the classifier's most predictive, non-redundant feature for source separability itself. The other four descriptors show weaker, closer-to-even sign splits across the interpretable cells and do not support a directional claim. We do not read this as evidence that any descriptor *causes* a reversal: because the sources are themselves observably separable (§4.6), an association between a descriptor and a reversal may reflect only a source-separating feature that co-occurs with where reversals were or were not observed. It is a candidate association worth a properly powered, purpose-built follow-up study; consistent with §3.3, this model is explanatory only and is never used to select, route, or recommend a scheduler.

9.6 Simulator Fidelity and Domain of Validity

§8's selected-case physical validation is informative primarily about a boundary of simulator fidelity, not as an independent statistical estimate of how often this benchmark's simulated findings would hold on real hardware in general (scope limits: §10). The non-reproduced reversal exposes a fidelity limitation specific to that selected comparison, while the stable control remained qualitatively stable – consistent with, but not a confirmation of, the simulator's broader findings. Together they show the same case-dependence documented in §6, now at the simulator-to-hardware boundary, without adding statistical weight to those earlier findings. This should not be read as an argument against simulation; it is an argument for stating the domain of validity of a simulation result and for using selected physical checks to identify where comparative simulation conclusions do, and do not, transfer.

9.7 Benchmark Construction and Practical Reproducibility

Regardless of outcome direction, this study's paired design, fixed scheduler panel, and preregistered protocol (§3) are intended to make the benchmark itself reusable: a future scheduler design can be dropped into the same panel and evaluated against the same frozen windows, operating regions, and metric contract, using the public artifact (§11) rather than re-deriving the protocol from this manuscript's prose.

10 Threats to Validity

10.1 Model Validity

Simulator abstraction. All results in this manuscript, except §8, are produced by a discrete-event simulator, not a real serving engine (§5.3). We do not claim that simulated absolute latency or throughput values match real-hardware values; only relative ranking and reversal direction were tested for transfer, on a small set of selected cases whose scope and outcome are detailed below.

Homogeneous hardware / execution assumptions. The simulator executes every cell under a fixed, single-GPU-colocated execution model for the PRIMARY panel; two disaggregation-requiring policies are confined to a secondary stratum and never pooled with the PRIMARY leaderboard (§5.1). Hardware effects such as kernel-level behavior, allocator overheads, network effects, and repeat-run hardware variability are abstracted away except in the selected physical validation.

Requests-per-window sensitivity. The benchmark corpus fixes window size at 200 requests throughout; how the reported results would change under a different requests-per-window choice has not been swept and remains an open item (§4.8 distinguishes this from the window-count analysis).

10.2 Implementation and Verification Validity

Fidelity of reimplemented policies. `FAITHFUL_EXTERNAL` policies are validated reimplementations, not the original authors' code; residual implementation gaps relative to the original published systems are possible despite unit-test validation (§5.2).

`STYLE_APPROXIMATION` *baselines.* The two style-level approximation policies are explicitly excluded from the PRIMARY headline panel and used only for a robustness check; they must not be read as official or validated reimplementations of their namesake systems (§5.2).

Verification scope. The campaign uses unit tests, deterministic repeated execution, schema checks, and structural validators (§5.4), but those checks do not prove that every implemented scheduler is identical to an external production system or that the simulator captures all hardware-level effects.

10.3 Statistical and Conclusion Validity

Zero completions and undefined completion-conditioned metrics. Several metrics (§5.7) are undefined, not zero, for any cell recording zero completions – a real methodological choice with a real cost: an undefined-metric policy is excluded from that specific condition's ranking-derived statistics, lowering effective sample size, and must be read alongside every ranking-derived table's reported exclusion count. This is not a rare edge case: 301 of 720 FIFO-only calibration-region records (§5.6) recorded zero completions, so a non-trivial share of (window, region) combinations contribute no completion-conditioned observation to §6's downstream analyses at that region for any policy – a structural constraint on statistical power, particularly in the highest-pressure regions.

Fixed thresholds are conventions, not universal standards. The 10% practical-margin (§5.10) and 0.9 recovery-probability (§7.2) thresholds used throughout this manuscript were both fixed prospectively, but neither is asserted to be a domain-standard or universally correct cutoff; every count this manuscript labels "practical" or "recovered" is conditional on the specific convention adopted, and a different reasonable choice could shift these counts without changing the underlying effect estimates.

SLO-definition sensitivity was a post-campaign extension, not preregistered evidence. An SLO-synthesis sensitivity check was planned but never actually preregistered before or during the benchmark campaign (§5.13); it was instead run afterward as an explicitly labeled post-campaign extension, after headline results were already known (§6.7). §6.7's non-trivial minority of SLO-sensitive reversals is a genuine, disclosed sensitivity, not a contradiction of any headline reversal, but a caveat that should temper confidence in the most SLO-definition-dependent individual comparisons.

10.4 External Validity

Three primary workload families. The benchmark corpus draws from three primary sources (§4.1); a fourth candidate source (TraceLab) is retained only as out-of-distribution context because its independence from a prior public window sweep cannot be fully verified. Every ranking-portability finding in §6 rests on these three sources alone – TraceLab contributes descriptor-level, out-of-distribution characterization context (§4.5) but no ranking-portability windows – and is not a claim about LLM-serving workloads in general.

Dataset/provider dependence. As established in §4.1, BurstGPT and this project's Azure traces are not established to come from independent underlying providers – both trace back to Microsoft Azure infrastructure, collected through different mechanisms. Any cross-source finding involving BurstGPT and Azure should be read with this in mind; Bailian/Qwen remains a genuinely independent platform.

Prompt-token semantic differences. Prompt-token fields compose differently across sources (single-request figure for BurstGPT versus potentially multi-turn-cumulative context for Azure and TraceLab; §4.4). Any prompt-length-based claim in this manuscript carries this caveat and should not be read as evidence that the sources' underlying conversations differ in length, only that their observed, request-level prompt-token figures do.

Unresolved historical byte-level overlap. An exact, byte-level reconstruction of a prior related project's specific 60-window public-trace replay set could not be independently re-derived from this project's available checkout; that project's own recorded parameters and final verdict are taken on the strength of direct manuscript inspection, not independently re-derived from local artifacts.

This project's own window construction is independently verified as non-identical (§4.3, §3.3), but the historical byte-level reconstruction gap itself remains an open item.

10.5 Construct and Metric Validity

Cross-metric disagreement bounds any single-metric ranking claim. §6.6's only-moderate cross-metric rank correlation and low top-policy agreement are a scope statement, not an analysis defect: the study's tracked metrics (ANWG, completion fraction, latency percentiles, SLO-violation rate, throughput) capture genuinely different deployment objectives, and no single one of them is privileged as "the" correct measure of scheduler quality beyond this study's own choice of ANWG as the PRIMARY headline metric (§5.7). Consequently, no ranking-portability claim in §6 should be read as a claim about scheduler comparisons in general; it is a claim about comparisons under the specific metric (or metrics) stated. The metric set evaluated here, while broader than a single-metric study, is not exhaustive of every metric a practitioner might care about, and cross-metric disagreement should itself be read as a substantive finding, not as noise to be averaged away.

10.6 Physical-Validation Scope

Selected-case validation only. §8's validation is limited by design to a small, frozen set of case-selected conditions (largest-effect-size reversal plus one stable-ordering control), not a full re-execution of the primary outcome matrix on real hardware; agreement or disagreement on those specific cases should not be over-generalized to every cell in §6. Concretely, the physical evaluation covers only 2 of the panel's 13 policies (`slai_faithful` and `vllm_faithful`), 3 of the workload sources, and a single operating region (`HIGH_PRESSURE`), on one fixed hardware/software environment (vLLM 0.27.1, a single A100 GPU allocation).

What the real-system confidence intervals mean. Uncertainty is quantified via window-level bootstrap resampling over 40 independent workload windows per source, using one physical execution per (policy, source, window) cell – the workload window, not repeated identical-hardware execution, is the unit over which statistical inference is drawn (§8.2). In particular, Table 4's confidence intervals capture *cross-window sampling uncertainty* – how much the effect would vary if a different set of 40 windows had been drawn – and not *repeat-run hardware variability*, since each (policy, source, window) cell was executed exactly once; no claim is made about how much a given cell's outcome would vary if that same cell were re-run on the same hardware. This result should not be read as a comprehensive validation of simulator fidelity, of every scheduler in the panel, or of every workload/region combination in §6.

11 Reproducibility and the LSSP Artifact

LSSP is released to support independent reproduction through linked public artifacts: a GitHub repository (https://github.com/SoroushVahidi/llm-serving-scheduler-robustness-benchmark) carrying the simulator, policy implementations, analysis and validation pipeline, figure/table generation code, manuscript source, and frozen provenance manifests for every campaign stage; a Hugging Face dataset (https://huggingface.co/datasets/SoroushVahidi/llm-serving-scheduler-portability) carrying the analysis-ready principal result tables and the reduced RQ6 result with computational provenance (§8.4); and a Zenodo snapshot, DOI 10.5281/zenodo.22306798. Upstream traces (§4.2) remain governed by their publishers' licenses, and several per-cell campaign records are not yet public; the full public/non-public release boundary is stated under Data Availability in the Declarations.

The primary comparative outcome (§6) was generated from, and structurally validated against, the full paired policy-outcome matrix across windows, regions, and metrics – including the per-cell mechanism telemetry (§5.9) – prior to any scientific interpretation. The primary statistical

analysis was executed once, deterministically, against the frozen 9,360-configuration matrix, and was independently structurally validated before any number in §6 or §7 was populated. The full validation record, scientific-freeze hashes, configuration and workload-manifest hashes, and per-cell seed/provenance fields are committed to the public repository's artifact documentation rather than reproduced in the main text.

The public artifacts are therefore sufficient to inspect the released principal result tables, reduced RQ6 analysis, frozen manifests, code, and provenance, but they are not a complete public dump of every intermediate or per-execution record generated by the campaign.

12 Conclusion

We have formulated the portability of a comparative scheduler ranking, $R(W, M, L)$, as an object of simulation-based scheduler selection and comparison, distinct from both workload characterization and single-scheduler evaluation, and designed, froze, and executed a paired, multi-source trace-driven simulation protocol – a mechanism-diverse fixed 13-policy scheduler panel, a calibrated six-region operating grid, an explicit metric contract, and an uncertainty/reversal-detection methodology – against 9,360 structurally validated configurations before interpreting any comparative outcome. Across three measurably different workload sources (§4), comparative rankings are portable in aggregate (Kendall's tau-b never below 0.55) but source-pair-dependent: two sources agree almost perfectly, while the third is consistently less portable against either at every operating region (§6). Supported practical reversals are uncommon (3.6% of pairwise comparisons) and concentrated: all involve the same policy pair and BurstGPT, at or above the pre-knee region (§6.3). Exact-order recovery of the full-corpus ranking requires most or all of the 40-window budget for two of three sources, whereas identifying the single best policy requires far fewer windows for those same two sources (§7).

Two post-campaign extensions show that portability is substantially weaker across evaluation metrics than across sources (median $\tau_b = 0.419$; §6.6) and only partly robust to the SLO definition (§6.7). RQ3 remains a pilot-scale engineering validation, not a transfer conclusion (§6.8). A selected physical vLLM/GPU validation (RQ6) did not reproduce the largest simulated reversal, while a paired stable control retained its qualitative ordering – a selected-case simulator-fidelity boundary, not a general estimate of hardware reversal prevalence (§8).

Ranking portability, in this simulation study, is therefore neither universal nor uniformly fragile: it is real, but conditional on the workload source, operating region, evaluation metric, SLO definition, and model fidelity context under which a comparison was established. The appropriate lesson is not to abandon simulation, but to report simulation-derived scheduler-selection claims with their experimental conditions and validity boundaries explicit. LSSP is released as a public, independently reproducible artifact (§11) so that each result can be verified and extended.

CRediT Authorship Contribution Statement

Soroush Vahidi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization.

Declaration of Competing Interest

The author declares no competing interests.

Funding

This work was supported in part by the Google Cloud Research Credits Program, which provided cloud credits used to support software development for this study; the CloudRift AI Builder Grant and the Cohere Labs Catalyst Grant Program, which provided cloud-computing and API credits.

Acknowledgements

The author gratefully acknowledges Professor Ioannis Koutis for his research guidance, continued support, and provision of computational resources that facilitated this work. Computational resources for this work were provided by Wulver, the high-performance computing cluster managed by Academic and Research Computing Systems (ARCS) in the Department of Information Services and Technology (IST) at the New Jersey Institute of Technology.

Data Availability

This study distinguishes four categories of data, consistent with §11. **Externally sourced public traces** (BurstGPT, Azure 2024, Bailian/Qwen) are governed by their original publishers' release terms and license status (§4.2; BurstGPT: CC-BY-4.0) and are not redistributed as part of this project's release; readers should obtain them from their original sources. **Derived benchmark manifests**, **reproduction scripts**, simulator code, policy implementations, and analysis code are available in the GitHub repository https://github.com/SoroushVahidi/llm-serving-scheduler-robustness-benchmark. The companion Hugging Face dataset repository https://huggingface.co/datasets/SoroushVahidi/llm-serving-scheduler-portability contains the current public, analysis-ready subset: source-license documentation, checksums, selected canonical result tables for the principal portability and sample-complexity results, and the reduced RQ6 analysis result with computational provenance. An immutable snapshot of the code repository at the version reported here is archived with a persistent identifier via Zenodo, DOI 10.5281/zenodo.22306798. The Hugging Face release is an initial, dataset-card-scoped publication (§11); it does *not* include the exploded per-cell campaign matrix (all 18,720 individual executions), complete per-cell mechanism telemetry, raw 240 per-cell RQ6 physical-execution records, or unpublished intermediate canonical-analysis files consumed by the figure/table generation script. These non-public records are candidates for a future dataset revision and are currently represented only by the released reduced tables, provenance records, and code.

Code Availability

The simulator, policy implementations, and analysis pipeline underlying this study are publicly available at https://github.com/SoroushVahidi/llm-serving-scheduler-robustness-benchmark, with an immutable archived snapshot via Zenodo, DOI 10.5281/zenodo.22306798.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the author used ChatGPT and Codex (OpenAI), Claude (Anthropic), Gemini (Google), Cursor, and Perplexity AI to assist with software development, debugging, experimental infrastructure, literature and research support, and manuscript drafting and editing. After using these tools, the author reviewed, verified, and revised all AI-assisted outputs, including code, analyses, references, and manuscript text, as needed and takes full responsibility for the content of the publication.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. VIDUR: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems (MLSys '24)*. MLSys. https://arxiv.org/abs/2405.05465

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In

Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24). USENIX Association. https://www.usenix.org/conference/osdi24/presentation/agrawal

[3] Alibaba Cloud. 2026. *qwen-bailian-usagetraces-anon*. Anonymized Bailian/Qwen production LLM-serving usage-trace release; repository `alibaba-edu/qwen-bailian-usagetraces-anon`, pinned commit `5f7439c51ec248a0c585f7d90a41a6f57773b912`, Apache-2.0.

[4] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. 2025. Optimal Scheduling Algorithms for LLM Inference: Theory and Practice. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 3 (2025), 59. doi:10.1145/3771574

[5] Zhongrui Chen, Adityo Anggraito, Diletta Olliaro, Andrea Marin, Marco Ajmone Marsan, Benjamin Berg, and Isaac Grosof. 2026. Improving nonpreemptive multiserver job scheduling with quickswap. *Performance Evaluation* 171 (2026), 102525. doi:10.1016/j.peva.2025.102525

[6] Jaehong Cho, Hyunmin Choi, and Jongse Park. 2025. LLMServingSim2.0: A Unified Simulator for Heterogeneous Hardware and Serving Techniques in LLM Infrastructure. *IEEE Computer Architecture Letters* 24, 2 (2025). https://arxiv.org/abs/2511.07229

[7] Jaehong Cho, Minsu Kim, Hyunmin Choi, Guseul Heo, and Jongse Park. 2024. LLMServingSim: A HW/SW Co-Simulation Infrastructure for LLM Inference Serving at Scale. In *Proceedings of the 2024 IEEE International Symposium on Workload Characterization (IISWC '24)*. IEEE. https://arxiv.org/abs/2408.05499

[8] Eric Ding, Byungsoo Oh, Bhaskar Kataria, Kaiwen Guo, Jelena Gvero, Abhishek Vijaya Kumar, Arjun Devraj, Lindsey Bowen, Atharv Sonwane, Emaad Manzoor, and Rachee Singh. 2026. CCL-Bench 1.0: A Trace-Based Benchmark for LLM Infrastructure. *arXiv preprint* (2026). https://arxiv.org/abs/2605.06544

[9] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS '24)*. https://arxiv.org/abs/2408.15792

[10] Shihong Gao, Xin Zhang, Yanyan Shen, and Lei Chen. 2025. Apt-Serve: Adaptive Request Scheduling on Hybrid Cache for Scalable LLM Inference Serving. *Proceedings of the ACM on Management of Data (PACMMOD)* 3, 3 (2025), 130. doi:10.1145/3725394

[11] Weizhe Huang, Tao Peng, Tongxuan Liu, Donghe Jin, Kang Meng, Xianzhe Dong, and Ke Zhang. 2025. ProServe: Unified Multi-Priority Request Scheduling for LLM Serving. *arXiv preprint* (2025). https://arxiv.org/abs/2512.12928

[12] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. In *Advances in Neural Information Processing Systems 36 (NeurIPS '23)*. doi:10.5555/3666122.3666913

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM. doi:10.1145/3600006.3613165

[14] Yueying Li, Yuanfan Chen, Jiayang Chen, Esha Choukse, Haoran Qiu, G. Edward Suh, Rodrigo Fonseca, Ziv Scully, and Udit Gupta. 2026. Beyond Prediction: Tail-Aware Scheduling for LLM Inference. In *Proceedings of the 43rd International Conference on Machine Learning (ICML '26)*. PMLR. https://arxiv.org/abs/2606.18431

[15] Yuhan Liu, Cong Xu, Qi Jia, Yihua Wang, Feiyu Chen, Liang Jin, Lu Liu, Yaqian Zhao, Yuting Ding, and Xiang Li. 2026. H-MAS: Hierarchical Multi-Agent Scheduling for Multi-Tenant LLM Serving. In *Findings of the Association for Computational Linguistics (ACL '26)*. Association for Computational Linguistics, 39051–39071. https://aclanthology.org/2026.findings-acl.1946/

[16] Microsoft Azure. 2024. *AzureLLMInferenceDataset2024*. https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2024.md

[17] William Nixon, Jon Durbin, Florian Standhartinger, Haryadi S. Gunawi, and Juncheng Yang. 2026. A Year in LLM Serving: Workload Evolution, Caching and Load-Balancing. *arXiv preprint* (2026). https://arxiv.org/abs/2608.13573

[18] Konstantinos Papaioannou and Thaleia Dimitra Doudali. 2024. The Importance of Workload Choice in Evaluating LLM Inference Systems. In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys '24)*. ACM, 39–46. doi:10.1145/3642970.3655823

[19] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE/ACM. doi:10.1109/ISCA59077.2024.00019

[20] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST '25)*. USENIX Association. https://arxiv.org/abs/2407.00079

[21] Chaoyi Ruan, Yinhe Chen, Dongqi Tian, Yandong Shi, Yongji Wu, Jialin Li, and Cheng Li. 2026. Libra: Flexible Request Partitioning and Scheduling for Serving Unbalanced and Dynamic LLM Workloads. In *Proceedings of the*

23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26). USENIX Association. https://www.usenix.org/conference/nsdi26/presentation/ruan-libra

[22] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2401.00588

[23] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *Proceedings of the 2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA '25)*. IEEE. https://arxiv.org/abs/2408.00741

[24] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2406.03243

[25] Tingyang Sun, Ting He, Bo Ji, and Parimal Parag. 2025. Optimizing resource allocation for geographically-distributed inference by large language models. *Performance Evaluation* 170 (2025), 102527. doi:10.1016/j.peva.2025.102527

[26] Yinghao Tang, Tingfeng Lan, Xiuqi Huang, Hui Lu, and Wei Chen. 2025. SCORPIO: Serving the Right Requests at the Right Time for Heterogeneous SLOs in LLM Inference. *arXiv preprint* (2025). https://arxiv.org/abs/2505.23022

[27] Yiheng Tao, Yihe Zhang, Matthew Dearing, Xin Wang, Yuping Fan, Michael E. Papka, and Zhiling Lan. 2025. Ranking Before Serving: Low-Latency LLM Serving via Pairwise Learning-to-Rank. *arXiv preprint* (2025). https://arxiv.org/abs/2510.03243

[28] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. *Proceedings of the 2025 USENIX Annual Technical Conference (ATC '25)* (2025). https://arxiv.org/abs/2506.02634

[29] Jiahao Wang, Kaizhan Lin, Kaixi Zhang, Jinbo Han, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Rong Chen, and Haibo Chen. 2026. SMetric: Rethink LLM Scheduling for Serving Agents with Balanced Session-centric Scheduling. *arXiv preprint* (2026). https://arxiv.org/abs/2607.08565

[30] Michael Wang, Yikang Yue, Shaobo Li, Yirui Eric Zhou, Chen Wang, and Jian Huang. 2026. Benchmarking LLM Serving Systems for Agentic AI Workloads with XPerf. *arXiv preprint* (2026). https://arxiv.org/abs/2608.20370

[31] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25), Volume 2*. ACM. doi:10.1145/3711896.3737413

[32] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2026. FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association. https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang

[33] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. 2026. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association. https://www.usenix.org/conference/nsdi26/presentation/xiang-servegen

[34] Mert Yildiz, Alexey Rolich, and Andrea Baiocchi. 2026. Dispatching policies in data center clusters: Insights from Google and Alibaba workloads. *Performance Evaluation* 172 (2026), 102551. doi:10.1016/j.peva.2026.102551

[35] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association. https://arxiv.org/abs/2207.00032

[36] Netanel Zakay and Dror G. Feitelson. 2014. Workload resampling for performance evaluation of parallel job schedulers. *Concurrency and Computation: Practice and Experience* 26, 12 (2014), 2079–2105. doi:10.1002/cpe.3240

[37] Dingyan Zhang, Jinbo Han, Kaixi Zhang, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Jingren Zhou, and Rong Chen. 2026. Simple is Better: Multiplication May Be All You Need for LLM Request Scheduling. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*. USENIX Association. https://arxiv.org/abs/2603.15202

[38] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. 2026. JITServe: SLO-aware LLM Serving with Imprecise Request Information. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association. https://www.usenix.org/conference/nsdi26/presentation/zhang-wei

[39] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2401.09670

[40] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. *arXiv preprint* (2026). https://arxiv.org/abs/2606.30560